• AUTONOMOUS PC AGENT & E-COMMERCE FORGE

JARVIS EPISODE 33 GUIDE

by Pulse of AI

▲ CRITICAL WARNING: MANDATORY PRE-REQUISITE

You must watch all the previous videos (Parts 1 to 32) and extract the baseline code and environment files directly from their pinned comments. If you do not configure the previous dependencies, JARVIS might not work!

01. COMPLETE SERIES ROADMAP (WATCH IN CHRONOLOGICAL ORDER)

MODULE	ENGINEERING CAPABILITY ADDED	DIRECT SOURCE
PART 1	Core Brain Setup & Instant Live API Integration	■ Watch Ep 1
PART 2	Autonomous System Expansion & Pipeline Upgrades	■ Watch Ep 2
PART 3	Autonomous System Expansion & Pipeline Upgrades	■ Watch Ep 3
PART 4	Autonomous System Expansion & Pipeline Upgrades	■ Watch Ep 4
PART 5	Autonomous System Expansion & Pipeline Upgrades	■ Watch Ep 5
PART 6	Autonomous System Expansion & Pipeline Upgrades	■ Watch Ep 6
PART 7	Autonomous Desktop Tool Calling Active	■ Watch Ep 7
PART 8	Autonomous System Expansion & Pipeline Upgrades	■ Watch Ep 8
PART 9	Autonomous System Expansion & Pipeline Upgrades	■ Watch Ep 9
PART 10	Autonomous System Expansion & Pipeline Upgrades	■ Watch Ep 10
PART 11	Autonomous System Expansion & Pipeline Upgrades	■ Watch Ep 11
PART 12	Autonomous System Expansion & Pipeline Upgrades	■ Watch Ep 12
PART 13	Autonomous System Expansion & Pipeline Upgrades	■ Watch Ep 13
PART 14	Autonomous System Expansion & Pipeline Upgrades	■ Watch Ep 14
PART 15	Autonomous System Expansion & Pipeline Upgrades	■ Watch Ep 15
PART 16	Autonomous System Expansion & Pipeline Upgrades	■ Watch Ep 16
PART 17	Autonomous System Expansion & Pipeline Upgrades	■ Watch Ep 17
PART 18	Autonomous System Expansion & Pipeline Upgrades	■ Watch Ep 18
PART 19	Autonomous System Expansion & Pipeline Upgrades	■ Watch Ep 19

PART 20	Autonomous System Expansion & Pipeline Upgrades	■ Watch Ep 20
PART 21	Autonomous System Expansion & Pipeline Upgrades	■ Watch Ep 21
PART 22	Autonomous System Expansion & Pipeline Upgrades	■ Watch Ep 22
PART 23	Autonomous Serious Mode Pipeline	■ Watch Ep 23
PART 24	Autonomous System Expansion & Pipeline Upgrades	■ Watch Ep 24
PART 25	Autonomous System Expansion & Pipeline Upgrades	■ Watch Ep 25
PART 26	Autonomous System Expansion & Pipeline Upgrades	■ Watch Ep 26
PART 27	Autonomous System Expansion & Pipeline Upgrades	■ Watch Ep 27
PART 28	Autonomous System Expansion & Pipeline Upgrades	■ Watch Ep 28
PART 29	Project Doppelgänger (Ghost Operator)	■ Watch Ep 29
PART 30	Project Sentinel (3D Holographic Globe HUD)	■ Watch Ep 30
PART 31	Project Schematic (3D Holograms)	■ Watch Ep 31
PART 32	Project EDITH (3D Neural Networks)	■ Watch Ep 32
PART 33	**Autonomous PC Agent & E-Commerce Forge**	■ Watch Ep 33

02. ENVIRONMENT ARCHITECTURE & SYSTEM CONFIG

[1] Terminal Dependencies:
pip install google-genai pyaudio geopy plotly PyQt6 PyQt6-WebEngine flask pywebview requests keyboard pillow

[2] Secure Environment File (.env):
GOOGLE_API_KEY=your_gemini_api_key

[3] Episode 33 Core Innovation:
Integration of a powerful, sandboxed Autonomous PC Agent that translates natural language commands into verified execution plans, generating production-ready code, manipulating workspace files, and launching applications like VS Code automatically.

03. PRODUCTION CODEBASE (DYNAMICALLY EXTRACTED)

The following source code has been extracted directly from your local project files.

// MODULE: pc_agent.py (AUTONOMOUS PC AGENT & WORKFLOW ORCHESTRATOR)

```
"""
pc_agent.py
JARVIS Autonomous PC Agent (Multi-Step System & Workflow Operator)
======================================================================
A powerful, secure, and verified computer automation engine that complements
and extends the Mark 85 Omni-Pilot.

Key Architecture:
  1. Agent Orchestrator: Multi-phase planning, state tracking, and recovery.
  2. Planner: Structured JSON plan generation via Gemini with strict schema validation.
  3. Tool Registry: Explicit allowlist of real, controlled PC tools.
  4. Safety Gatekeeper: Path containment, approval binding, and hardcoded blocklists.
  5. State Verifier: Checks real file contents, exit codes, and process output.
  6. Cancellation: Emergency [ESC] polling, thread abort, and child process termination.
  7. Telemetry: Full-duplex live event streaming to Holographic FUI & Tkinter HUD.

Built by Swayam Dhawale — Founder of Pulse of AI.
"""

import os
import sys
import re
import json
import time
import shutil
import hashlib
import pathlib
import threading
import subprocess
from typing import Dict, Any, List, Optional, Tuple

import keyboard
from PIL import Image
from dotenv import load_dotenv
from rich.console import Console

# Import existing JARVIS UI, Audio & Vision Primitives
import fui_bridge
import screen_clicker
from desktop_pilot import (
    play_pilot_start_sfx,
    play_pilot_step_sfx,
    play_pilot_complete_sfx,
    play_pilot_abort_sfx,
    _execute_click,
    _execute_type,
    _execute_hotkey,
    _execute_scroll,
    _execute_launch_app,
    _execute_open_url,
    _check_emergency_abort,
)

load_dotenv()
console = Console()

# Windows UTF-8 stdout configuration
if sys.platform == "win32":
    try:
        sys.stdout.reconfigure(encoding="utf-8")
        sys.stderr.reconfigure(encoding="utf-8")
```

```
    except Exception:
        pass

# API Client Initialization via existing keyring Setup
try:
    from Setup import get_or_prompt_api_key
    API_KEY = get_or_prompt_api_key()
except Exception:
    API_KEY = os.getenv("GOOGLE_API_KEY")

from google import genai
client = genai.Client(api_key=API_KEY) if API_KEY else genai.Client()

# Global State Control
_agent_lock = threading.Lock()
_agent_active = False
_abort_requested = False
_current_task = ""
_active_process: Optional[subprocess.Popen] = None
_approved_tokens: set = set()

# Configurable Approved Workspace
PROJECT_ROOT = os.path.dirname(os.path.abspath(__file__))
DEFAULT_WORKSPACE_DIR = os.path.join(PROJECT_ROOT, "workspace")

# Protected Paths & Files that can NEVER be touched or traversed into
PROTECTED_PATHS = [
    os.path.expandvars(r"%SystemRoot%"),
    os.path.expandvars(r"%ProgramFiles%"),
    os.path.expandvars(r"%ProgramFiles(x86)%"),
    os.path.join(PROJECT_ROOT, ".env"),
    os.path.join(PROJECT_ROOT, ".git"),
    os.path.join(PROJECT_ROOT, "backend"),
]

def _safe_log(text: str, style: str = ""):
    try:
        if style:
            console.print(f"[{style}]{text}[/{style}]")
        else:
            console.print(text)
    except Exception:
        clean = text.encode("ascii", "replace").decode("ascii")
        print(clean)

def get_workspace_dir() -> str:
    """Returns the validated active workspace directory, creating it if needed."""
    ws = os.getenv("JARVIS_WORKSPACE_DIR", DEFAULT_WORKSPACE_DIR)
    os.makedirs(ws, exist_ok=True)
    return os.path.abspath(ws)

def is_agent_active() -> bool:
    """Returns True if the Autonomous PC Agent is actively executing."""
    return _agent_active

def abort_pc_agent() -> str:
    """
    Emergency halt signal for the Autonomous PC Agent.
    Terminates active child processes, clears pending plans, and restores manual control.
    """
    global _abort_requested, _agent_active, _active_process
    _abort_requested = True
    _agent_active = False

    # Kill any active owned child process
    if _active_process is not None:
        try:
            _active_process.kill()
```

```
        except Exception:
            pass
        _active_process = None

    play_pilot_abort_sfx()
    _safe_log("■ [PC AGENT] Execution halted by user command or [ESC] key.", "bold red")
    fui_bridge.broadcast_pc_agent_status("aborted", 0, 0, "ABORTED BY USER", _current_task, "Mission halted. Manual
control restored.")
    fui_bridge.broadcast_log("■ [PC AGENT] Emergency stop disengaged. Manual control restored.", "system")
    return "Autonomous PC Agent has been aborted, sir. All processes halted."

def check_abort() -> bool:
    """Checks whether cancellation was requested via ESC key or abort signal."""
    global _abort_requested
    if _abort_requested or keyboard.is_pressed("esc"):
        _abort_requested = True
        return True
    return False


# ============================================================================
# 1. SECURITY & PATH CONTAINMENT GATEKEEPER
# ============================================================================

def validate_workspace_path(path_str: str, allow_create_parent: bool = True) -> Tuple[bool, str]:
    """
    Validates that a path is strictly inside the approved workspace.
    Blocks directory traversal ('..'), symlink escapes, and system roots.
    Returns (is_valid, resolved_absolute_path_or_error_message).
    """
    workspace = get_workspace_dir()
    try:
        # If relative, resolve against workspace
        if not os.path.isabs(path_str):
            target = os.path.normpath(os.path.join(workspace, path_str))
        else:
            target = os.path.normpath(path_str)

        target_real = os.path.realpath(target)
        workspace_real = os.path.realpath(workspace)

        # Check containment (case-insensitive on Windows)
        if sys.platform == "win32":
            contained = target_real.lower().startswith(workspace_real.lower())
        else:
            contained = target_real.startswith(workspace_real)

        if not contained:
            return False, f"Access Denied: Path '{path_str}' escapes approved workspace '{workspace}'."

        # Check protected paths
        for prot in PROTECTED_PATHS:
            prot_real = os.path.realpath(prot)
            is_prot = (target_real.lower().startswith(prot_real.lower())) if sys.platform == "win32" else
target_real.startswith(prot_real)
            if is_prot:
                return False, f"Access Denied: Path '{path_str}' points to a protected system resource."

        return True, target_real
    except Exception as e:
        return False, f"Path validation error: {str(e)}"

def generate_approval_token(task_id: str, tool_name: str, args: dict) -> str:
    """Generates an immutable cryptographic hash binding approval to exact tool and arguments."""
    payload = f"{task_id}:{tool_name}:{json.dumps(args, sort_keys=True)}"
    return hashlib.sha256(payload.encode("utf-8")).hexdigest()

def register_approval(token: str):
```

```
    """Registers an approved token."""
    _approved_tokens.add(token)

def is_approved(token: str) -> bool:
    """Checks whether an exact token was approved."""
    return token in _approved_tokens


# ============================================================================
# 2. REAL CONTROLLED PC TOOL REGISTRY
# ============================================================================

def tool_get_pc_info(args: dict) -> dict:
    """Safe, read-only system telemetry and environment audit."""
    ws = get_workspace_dir()

    # Detect Python
    python_path = sys.executable
    python_ver = f"{sys.version_info.major}.{sys.version_info.minor}.{sys.version_info.micro}"

    # Detect VS Code
    vscode_path = shutil.which("code") or shutil.which("code.cmd")

    # Disk usage in workspace
    try:
        total, used, free = shutil.disk_usage(ws)
        disk_free_gb = round(free / (1024**3), 2)
    except Exception:
        disk_free_gb = "Unknown"

    return {
        "status": "success",
        "os": sys.platform,
        "workspace": ws,
        "python_available": True,
        "python_path": python_path,
        "python_version": python_ver,
        "vscode_available": bool(vscode_path),
        "vscode_path": vscode_path or "Not Found on PATH",
        "disk_free_gb": disk_free_gb,
    }

def tool_open_app(args: dict) -> dict:
    """
    Controlled application launcher with allowlist.
    Prevents arbitrary shell injection.
    """
    app_key = args.get("app", "").lower().strip()
    target_path = args.get("path", "").strip()

    # Allowlist of safe applications
    ALLOWLIST_APPS = {
        "code": "code.cmd" if sys.platform == "win32" else "code",
        "vscode": "code.cmd" if sys.platform == "win32" else "code",
        "notepad": "notepad.exe",
        "calc": "calc.exe",
        "calculator": "calc.exe",
        "explorer": "explorer.exe",
    }

    if app_key not in ALLOWLIST_APPS:
        return {
            "status": "error",
            "message": f"Application '{app_key}' is not in the controlled allowlist: {list(ALLOWLIST_APPS.keys())}"
        }

    exe = ALLOWLIST_APPS[app_key]
    cmd = [exe]
```

```
    if target_path:
        valid, resolved = validate_workspace_path(target_path)
        if not valid:
            return {"status": "error", "message": resolved}
        cmd.append(resolved)

    try:
        # Use CREATE_NO_WINDOW or normal process launch without shell
        subprocess.Popen(cmd, shell=False)
        return {
            "status": "success",
            "message": f"Successfully launched {app_key} with target '{target_path or 'default'}'."
        }
    except Exception as e:
        return {"status": "error", "message": f"Failed to launch '{app_key}': {str(e)}"}

def tool_create_project_folder(args: dict) -> dict:
    """Creates a project folder strictly inside approved workspace."""
    folder_name = args.get("folder_name", "").strip()
    if not folder_name:
        return {"status": "error", "message": "Missing 'folder_name' parameter."}

    valid, target_path = validate_workspace_path(folder_name)
    if not valid:
        return {"status": "error", "message": target_path}

    if os.path.exists(target_path):
        return {
            "status": "success",
            "already_existed": True,
            "path": target_path,
            "message": f"Project folder already exists at '{target_path}'."
        }

    try:
        os.makedirs(target_path, exist_ok=True)
        if os.path.isdir(target_path):
            return {
                "status": "success",
                "path": target_path,
                "message": f"Successfully created project folder at '{target_path}'."
            }
        else:
            return {"status": "error", "message": "Verification failed: folder was not created."}
    except Exception as e:
        return {"status": "error", "message": f"Failed to create folder: {str(e)}"}

def tool_write_workspace_file(args: dict, task_id: str = "") -> dict:
    """
    Writes content to a file inside the approved workspace.
    Guards against overwriting without explicit approval.
    """
    file_path = args.get("file_path", "").strip()
    content = args.get("content", "")
    overwrite = args.get("overwrite", False)

    if not file_path:
        return {"status": "error", "message": "Missing 'file_path' parameter."}

    valid, target_path = validate_workspace_path(file_path)
    if not valid:
        return {"status": "error", "message": target_path}

    # Overwrite Guard
    if os.path.exists(target_path) and not overwrite:
        token = generate_approval_token(task_id, "write_workspace_file", args)
        if not is_approved(token):
```

```
            return {
                "status": "needs_approval",
                "approval_token": token,
                "message": f"File '{file_path}' already exists. Overwrite requires explicit confirmation.",
                "file_path": target_path
            }

    try:
        parent_dir = os.path.dirname(target_path)
        os.makedirs(parent_dir, exist_ok=True)

        with open(target_path, "w", encoding="utf-8") as f:
            f.write(content)

        # Verification: Read back bytes and size
        if os.path.exists(target_path):
            written_size = os.path.getsize(target_path)
            return {
                "status": "success",
                "file_path": target_path,
                "bytes_written": written_size,
                "message": f"Verified write: {written_size} bytes written to '{os.path.basename(target_path)}'."
            }
        else:
            return {"status": "error", "message": "Verification failed: file was not saved."}
    except Exception as e:
        return {"status": "error", "message": f"Failed to write file: {str(e)}"}

def tool_read_workspace_file(args: dict) -> dict:
    """Reads a file inside approved workspace with size limit (max 512KB)."""
    file_path = args.get("file_path", "").strip()
    valid, target_path = validate_workspace_path(file_path)
    if not valid:
        return {"status": "error", "message": target_path}

    if not os.path.isfile(target_path):
        return {"status": "error", "message": f"File does not exist: '{file_path}'."}

    try:
        size = os.path.getsize(target_path)
        if size > 512 * 1024:
            return {"status": "error", "message": f"File exceeds maximum read limit of 512KB ({size} bytes)."}

        with open(target_path, "r", encoding="utf-8", errors="replace") as f:
            content = f.read()

        return {
            "status": "success",
            "file_path": target_path,
            "size": size,
            "content": content
        }
    except Exception as e:
        return {"status": "error", "message": f"Failed to read file: {str(e)}"}

def tool_run_dev_command(args: dict, task_id: str = "") -> dict:
    """
    Executes an approved development command with strict argument structures,
    timeouts, output bounding, and process cancellation.
    """
    global _active_process
    command = args.get("command", "").strip()
    cmd_args = args.get("args", [])
    cwd_rel = args.get("cwd", "")
    timeout_sec = min(60, max(5, int(args.get("timeout", 25))))

    # Allowlist of permitted command executables
    ALLOWED_COMMANDS = {
```

```
        "python": sys.executable,
        "python3": sys.executable,
        "node": shutil.which("node"),
        "git": shutil.which("git"),
    }

    if command.lower() not in ALLOWED_COMMANDS:
        return {
            "status": "error",
            "message": f"Command '{command}' is not in the development allowlist: {list(ALLOWED_COMMANDS.keys())}"
        }

    executable = ALLOWED_COMMANDS[command.lower()]
    if not executable or not os.path.exists(executable):
        return {
            "status": "error",
            "message": f"Executable for '{command}' was not found on this system."
        }

    # Validate working directory
    valid, target_cwd = validate_workspace_path(cwd_rel if cwd_rel else ".")
    if not valid:
        return {"status": "error", "message": target_cwd}

    # Build safe execution vector
    exec_vector = [executable] + [str(a) for a in cmd_args]

    # Consequential / Shell Injection Defense:
    # Any chained shell character in args is strictly prohibited
    for a in cmd_args:
        if any(c in str(a) for c in [";", "&&", "||", "|", "`", "$("]):
            return {
                "status": "error",
                "message": f"Security Violation: Shell metacharacter detected in argument '{a}'."
            }

    try:
        _safe_log(f" [EXEC] Running: {command} {' '.join(cmd_args)} in {target_cwd}", "cyan")
        _active_process = subprocess.Popen(
            exec_vector,
            cwd=target_cwd,
            stdout=subprocess.PIPE,
            stderr=subprocess.PIPE,
            text=True,
            shell=False,
        )

        try:
            stdout, stderr = _active_process.communicate(timeout=timeout_sec)
            exit_code = _active_process.returncode
        except subprocess.TimeoutExpired:
            _active_process.kill()
            _active_process = None
            return {
                "status": "error",
                "timed_out": True,
                "message": f"Process timed out after {timeout_sec} seconds and was terminated.",
                "exit_code": -1
            }

        _active_process = None

        # Truncate output to prevent massive buffers
        max_chars = 4000
        out_trunc = stdout[:max_chars] if stdout else ""
        err_trunc = stderr[:max_chars] if stderr else ""

        return {
```

```
            "status": "success" if exit_code == 0 else "failed",
            "exit_code": exit_code,
            "stdout": out_trunc,
            "stderr": err_trunc,
            "message": f"Process exited with code {exit_code}."
        }
    except Exception as e:
        _active_process = None
        return {"status": "error", "message": f"Execution failed: {str(e)}"}

def tool_inspect_project(args: dict) -> dict:
    """Lists files and folders in approved workspace with recursion limit."""
    rel_path = args.get("path", "")
    valid, target_path = validate_workspace_path(rel_path)
    if not valid:
        return {"status": "error", "message": target_path}

    if not os.path.exists(target_path):
        return {"status": "error", "message": f"Path does not exist: '{rel_path}'."}

    results = []
    max_items = 60
    for root, dirs, files in os.walk(target_path):
        # Exclude hidden and cache folders
        dirs[:] = [d for d in dirs if not d.startswith(".") and d not in ["__pycache__", "node_modules"]]
        for f in files:
            if f.startswith(".") or f == ".env":
                continue
            full = os.path.join(root, f)
            rel = os.path.relpath(full, target_path)
            results.append({"name": rel, "size": os.path.getsize(full), "type": "file"})
            if len(results) >= max_items:
                break
        if len(results) >= max_items:
            break

    return {
        "status": "success",
        "path": target_path,
        "item_count": len(results),
        "items": results
    }

def tool_verify_step(args: dict) -> dict:
    """
    Verification layer helper.
    Asserts expected outcome: file existence, string matching, exit codes.
    Normalizes paths, quotes (' vs " vs smart-quotes), line-endings (CRLF vs LF),
    and whitespace so verification is robust while truthfully catching actual content mismatches.
    """
    check_type = args.get("type", "").lower().strip()
    target = args.get("target", "").strip()
    expected = args.get("expected", "")

    # Normalize target path separators
    if target:
        target = os.path.normpath(target).replace("\\", "/")

    if check_type in ["file_exists", "folder_exists", "dir_exists", "path_exists"]:
        valid, resolved = validate_workspace_path(target)
        exists = valid and os.path.exists(resolved)
        return {
            "status": "success" if exists else "failed",
            "verified": exists,
            "message": f"Path '{target}' exists." if exists else f"Path '{target}' does NOT exist."
        }

    elif check_type in ["file_contains", "file_has_content", "content_matches"]:
```

```
        valid, resolved = validate_workspace_path(target)
        if not valid or not os.path.isfile(resolved):
            return {
                "status": "failed",
                "verified": False,
                "message": f"File '{target}' does not exist."
            }

        with open(resolved, "r", encoding="utf-8", errors="replace") as f:
            content = f.read()

        # 1. Direct exact match
        match = expected in content

        # 2. Line-ending normalization (CRLF vs LF)
        if not match:
            c_norm = content.replace("\r\n", "\n")
            e_norm = expected.replace("\r\n", "\n")
            match = e_norm in c_norm

            # 3. Quote and smart-quote normalization (' vs " vs “ vs ” vs ‘ vs ’)
            if not match:
                def _norm_quotes(s: str) -> str:
                    return s.replace("'", '"').replace("“", '"').replace("”", '"').replace("‘", '"').replace("’",
'"')
                c_quotes = _norm_quotes(c_norm)
                e_quotes = _norm_quotes(e_norm)
                match = (e_quotes in c_quotes) or (e_quotes.strip() in c_quotes)

            # 4. Whitespace-trimmed multi-line statement check
            if not match and "\n" in e_norm:
                e_lines = [l.strip() for l in e_norm.split("\n") if l.strip()]
                c_lines = [l.strip() for l in c_norm.split("\n") if l.strip()]
                if e_lines and all(any(el in cl for cl in c_lines) for el in e_lines):
                    match = True

            # 5. Non-empty string stripped check
            if not match and expected.strip():
                match = expected.strip() in content

        if match:
            return {
                "status": "success",
                "verified": True,
                "message": f"File '{target}' contains expected content."
            }
        else:
            exp_preview = repr(expected[:50]) if expected else "''"
            act_preview = repr(content[:50]) if content else "''"
            return {
                "status": "failed",
                "verified": False,
                "message": f"Substring not found in '{target}'. Expected: {exp_preview}, Actual: {act_preview}."
            }

    elif check_type in ["exit_code", "process_exit_code"]:
        actual_code = args.get("actual_code", 0)
        try:
            expected_code = int(str(expected).strip())
            passed = (actual_code == expected_code)
        except Exception:
            passed = (str(actual_code) == str(expected).strip())
        return {
            "status": "success" if passed else "failed",
            "verified": passed,
            "message": f"Exit code is {actual_code} (expected {expected})."
        }
```

```
    elif check_type in ["output_contains", "stdout_contains"]:
        actual_out = args.get("actual_output", "")
        # Exact match
        match = expected in actual_out
        if not match:
            # Normalize line endings and quotes
            a_norm = actual_out.replace("\r\n", "\n")
            e_norm = expected.replace("\r\n", "\n")
            match = (e_norm in a_norm) or (e_norm.strip() in a_norm)
            if not match:
                def _norm_quotes(s: str) -> str:
                    return s.replace("'", '"').replace("“", '"').replace("”", '"').replace("‘", '"').replace("’",
'"')
                match = _norm_quotes(e_norm).strip() in _norm_quotes(a_norm)

        return {
            "status": "success" if match else "failed",
            "verified": match,
            "message": f"Captured output contains '{expected}'." if match else f"Expected substring '{expected}' not
in output."
        }

    return {"status": "error", "message": f"Unknown verification type: '{check_type}'."}

# Master Tool Registry
TOOL_REGISTRY = {
    "get_pc_info": tool_get_pc_info,
    "open_app": tool_open_app,
    "create_project_folder": tool_create_project_folder,
    "write_workspace_file": tool_write_workspace_file,
    "read_workspace_file": tool_read_workspace_file,
    "run_dev_command": tool_run_dev_command,
    "inspect_project": tool_inspect_project,
    "verify_step": tool_verify_step,
}


# ==========================================================================
# 3. STRUCTURED PLANNER & ORCHESTRATION ENGINE
# ==========================================================================

PLANNER_PROMPT_TEMPLATE = """
You are the JARVIS Autonomous PC Agent Planner.
Your role is to translate a user objective into an ordered, verified multi-step plan using ONLY permitted tools.

Permitted Tools and Arguments:
1. get_pc_info: {{}}
2. open_app: {{"app": "code"|"notepad"|"calc", "path": "optional relative path inside workspace"}}
3. create_project_folder: {{"folder_name": "folder name inside workspace"}}
4. write_workspace_file: {{"file_path": "path inside workspace", "content": "file contents", "overwrite": true}}
5. read_workspace_file: {{"file_path": "path inside workspace"}}
6. run_dev_command: {{"command": "python", "args": ["script.py"], "cwd": "relative folder inside workspace",
"timeout": 25}}
7. inspect_project: {{"path": "relative folder"}}
8. verify_step: {{"type": "file_exists"|"file_contains"|"exit_code"|"output_contains", "target": "relative path",
"expected": "string"}}

Rules:
1. Return ONLY a valid JSON object matching the schema below.
2. Every major action MUST be followed by a verification step.
3. For project creation tasks (e.g. 'Create a Python project, write its main file, open it in VS Code, run it, and
verify the output' or 'NeuralCore'):
   - First call get_pc_info to verify Python and VS Code availability.
   - Next create the project folder.
   - Write the Python code into the project folder (e.g. main.py).
   - Verify the file exists and contains content.
   - Launch in VS Code (open_app with app="code" and path set to project folder).
   - Run the Python script (run_dev_command with command="python", args=["main.py"], cwd=project folder).
```

```
   - Verify exit code and output.
4. Do NOT output markdown code fences or conversational text. Output raw JSON.

JSON Schema:
{{
  "task_id": "task_12345",
  "objective": "Brief summary",
  "steps": [
    {{
      "step_num": 1,
      "description": "Human-readable description of what will be done",
      "tool": "tool_name",
      "args": {{ ... }},
      "expected_outcome": "What success looks like",
      "is_consequential": false
    }}
  ]
}}
"""

def generate_plan(user_goal: str) -> Optional[dict]:
    """Generates and validates a structured execution plan using Gemini."""
    prompt = f"{PLANNER_PROMPT_TEMPLATE}\n\nUser Objective: \"{user_goal}\""

    for model_name in ["gemini-3.8-flash", "gemini-3.6-flash"]:
        try:
            resp = client.models.generate_content(
                model=model_name,
                contents=prompt
            )
            if resp and resp.text:
                raw = resp.text.strip()
                if raw.startswith("```"):
                    raw = re.sub(r"^```[a-zA-Z]*\n?", "", raw)
                    raw = re.sub(r"\n?```$", "", raw).strip()
                data = json.loads(raw)

                # Plan Schema Validation Gate
                if isinstance(data, dict) and "steps" in data and isinstance(data["steps"], list):
                    # Validate all tools exist
                    valid_tools = True
                    for s in data["steps"]:
                        t = s.get("tool")
                        if t not in TOOL_REGISTRY:
                            _safe_log(f" [PLAN REJECTED] Unknown tool '{t}' proposed by model.", "yellow")
                            valid_tools = False
                            break
                    if valid_tools:
                        return data
        except Exception as e:
            _safe_log(f"[PLANNER RETRY] {model_name} error: {e}", "dim yellow")
            continue

    # Fallback to router
    try:
        import llm_router
        if llm_router:
            r_res = llm_router.generate_with_best_model(prompt)
            if r_res and r_res.get("text"):
                raw = r_res["text"].strip()
                if raw.startswith("```"):
                    raw = re.sub(r"^```[a-zA-Z]*\n?", "", raw)
                    raw = re.sub(r"\n?```$", "", raw).strip()
                data = json.loads(raw)
                if isinstance(data, dict) and "steps" in data and isinstance(data["steps"], list):
                    return data
    except Exception:
        pass
```

```
    return None

def _is_explicit_hello_world(task: str) -> bool:
    """Returns True ONLY if user explicitly and specifically requests 'Hello World'."""
    t = task.lower().strip()
    return "hello world" in t or "hello, world" in t or "minimal introductory example" in t

def _is_coding_request(task: str) -> bool:
    """Identifies if a user prompt is requesting software engineering, coding, script writing, or web/app
development."""
    t = task.lower().strip()
    if _is_explicit_hello_world(t):
        return True

    keywords = [
        "code", "script", "program", "app", "application", "calculator", "game",
        "website", "web", "html", "css", "javascript", "python", "api", "backend",
        "frontend", "bot", "chatbot", "ui", "gui", "function", "class", "method", "algorithm",
        "bug", "fix", "feature", "refactor", "project", "tool", "develop", "write",
        "create", "build", "make", "implement", "banao", "likho", "[redacted]", "[redacted]",
        "[redacted]", "banaye", "karo", "kuch", "koi", "main.py", "test", "tests",
        ".py", ".html", ".css", ".js", ".json", ".ts", ".jsx", ".tsx", ".java", ".cpp"
    ]
    return any(k in t for k in keywords)

def _inspect_workspace_context(task: str) -> dict:
    """
    Inspects existing workspace projects and files to support follow-up modifications,
    bug fixes, and feature additions without breaking or recreating projects from scratch.
    """
    ws = get_workspace_dir()
    context = {"workspace": ws, "files": {}, "existing_projects": []}

    try:
        task_lower = task.lower()
        is_mod = any(w in task_lower for w in ["modify", "add", "update", "fix", "change", "refactor", "extend",
"edit", "improve"])

        for entry in os.listdir(ws):
            full_path = os.path.join(ws, entry)
            if os.path.isdir(full_path) and not entry.startswith("."):
                context["existing_projects"].append(entry)
                # If directory name appears in task, or if task is a modification and this is a recent project
                if entry.lower() in task_lower or is_mod:
                    for root, _, files in os.walk(full_path):
                        for f in files:
                            if f.endswith((".py", ".html", ".css", ".js", ".json", ".md", ".txt")) and not
f.startswith("."):
                                f_full = os.path.join(root, f)
                                rel = os.path.relpath(f_full, ws).replace("\\", "/")
                                if os.path.getsize(f_full) < 100 * 1024:
                                    try:
                                        with open(f_full, "r", encoding="utf-8", errors="replace") as fp:
                                            context["files"][rel] = fp.read()
                                    except Exception:
                                        pass
                                if len(context["files"]) >= 10:
                                    break
            elif os.path.isfile(full_path) and not entry.startswith("."):
                if entry.lower() in task_lower or is_mod:
                    if os.path.getsize(full_path) < 100 * 1024:
                        try:
                            with open(full_path, "r", encoding="utf-8", errors="replace") as fp:
                                context["files"][entry] = fp.read()
                        except Exception:
                            pass
    except Exception as e:
```

```
        _safe_log(f" [CONTEXT WARNING] Error reading workspace context: {e}", "dim yellow")

    return context

def _extract_json_payload(raw_text: str) -> Optional[dict]:
    """Robust JSON extraction from LLM response, stripping code blocks and comments."""
    raw = raw_text.strip()
    if "```json" in raw:
        raw = raw.split("```json")[1].split("```")[0].strip()
    elif "```" in raw:
        raw = raw.split("```")[1].split("```")[0].strip()

    try:
        data = json.loads(raw)
        if isinstance(data, dict) and "files" in data and isinstance(data["files"], list):
            return data
    except Exception:
        pass

    m = re.search(r"(\{.*\})", raw, re.DOTALL)
    if m:
        try:
            data = json.loads(m.group(1))
            if isinstance(data, dict) and "files" in data and isinstance(data["files"], list):
                return data
        except Exception:
            pass

    return None

def generate_task_code(task: str, workspace_context: Optional[dict] = None) -> Optional[dict]:
    """
    Dynamic, task-aware code generator for JARVIS Autonomous PC Agent.
    Generates real, complete, production-quality code for arbitrary user requests in any language.
    NEVER returns hardcoded demo strings or placeholders unless explicitly requested.
    """
    task_clean = task.strip()
    is_hw = _is_explicit_hello_world(task_clean)

    if is_hw:
        return {
            "project_name": "hello_world",
            "language": "python",
            "files": [
                {
                    "path": "main.py",
                    "content": 'print("Hello, World!")\n'
                }
            ],
            "entry_point": "main.py",
            "run_command": {"command": "python", "args": ["main.py"]},
            "verification_markers": ["Hello, World!"],
            "features_implemented": ["Minimal Python Hello World program requested by user"]
        }

    # Build context string if existing project files are relevant
    context_str = ""
    if workspace_context and workspace_context.get("files"):
        context_str = "\nEXISTING WORKSPACE FILES (Preserve all existing working functions/classes unless asked to
replace):\n"
        for f_path, f_content in workspace_context["files"].items():
            context_str += f"=== Existing File: {f_path} ===\n{f_content[:2500]}\n"

    system_prompt = (
        "You are the JARVIS Autonomous PC Software Engineer.\n"
        "Your task is to write real, complete, production-grade code implementing the user's exact objective.\n"
        "CRITICAL RULES:\n"
        "1. Write the ACTUAL complete code requested (e.g. calculator must implement full arithmetic operations,
```

```
game must implement game loop, website must have HTML/CSS/JS).\n"
        "2. For CLI or testable programs, include an automated self-test or sample execution in the main block so it
runs cleanly when executed.\n"
        "3. NEVER output placeholders, dummy text, or '// TODO: Implement'.\n"
        "4. If modifying an existing project, preserve all working code and seamlessly integrate the requested
addition.\n"
        "5. Output ONLY a valid JSON object matching the schema below. No markdown fences, no conversational
prose.\n\n"
        "JSON SCHEMA:\n"
        "{\n"
        " \"project_name\": \"clean_folder_name\",\n"
        " \"language\": \"python\" | \"html\" | \"javascript\" | \"node\" | \"other\",\n"
        " \"files\": [\n"
        " {\"path\": \"relative/path/inside/project/file.ext\", \"content\": \"complete working code\"}\n"
        " ],\n"
        " \"entry_point\": \"relative/path/inside/project/main_file.ext\",\n"
        " \"run_command\": {\"command\": \"python\", \"args\": [\"main.py\"]},\n"
        " \"verification_markers\": [\"expected_keyword_or_func1\", \"expected_keyword_or_func2\"],\n"
        " \"features_implemented\": [\"Feature 1 description\", \"Feature 2 description\"]\n"
        "}"
    )

    user_prompt = f"User Request: \"{task_clean}\"\n{context_str}"

    # Try Gemini Flash models first
    for model_name in ["gemini-3.8-flash", "gemini-3.6-flash"]:
        try:
            resp = client.models.generate_content(
                model=model_name,
                contents=f"{system_prompt}\n\n{user_prompt}"
            )
            if resp and resp.text:
                parsed = _extract_json_payload(resp.text)
                if parsed and parsed.get("files"):
                    return parsed
        except Exception as e:
            _safe_log(f" [CODEGEN RETRY] {model_name}: {e}", "dim yellow")
            continue

    # Fallback to Groq via llm_router
    try:
        import llm_router
        if llm_router:
            router_res = llm_router.generate_with_best_model(user_prompt, system_prompt=system_prompt)
            if router_res and router_res.get("text"):
                parsed = _extract_json_payload(router_res["text"])
                if parsed and parsed.get("files"):
                    return parsed
    except Exception as e:
        _safe_log(f" [CODEGEN RETRY] Router fallback failed: {e}", "dim yellow")

    return None

def execute_dynamic_coding_workflow(task: str, workspace_dir: str = None) -> str:
    """
    Real, Task-Aware, General-Purpose Autonomous Coding Workflow for JARVIS.
    Understands user intent, programming language, and requirements.
    Inspects existing workspace context for follow-ups and modifications.
    Generates real, complete implementation code using the multi-tier LLM engine.
    Writes files to approved workspace, verifies syntax, markers, and execution.
    Launches project in VS Code and truthfully reports all outcomes and timing metrics.
    NEVER substitutes hardcoded templates or dummy Hello World text.
    """
    start_time = time.perf_counter()
    task_clean = task.strip()
    is_hw = _is_explicit_hello_world(task_clean)

    fui_bridge.broadcast_pc_agent_status("planning", 0, 5, "Analyzing software requirements and architecture...",
task_clean)
```

```
    fui_bridge.broadcast_log(f"■ [PC AGENT] Engineering real code solution for: '{task_clean}'...", "system")
    play_pilot_start_sfx()

    # Step 1: Context Inspection
    ws_ctx = _inspect_workspace_context(task_clean)
    if ws_ctx.get("files"):
        _safe_log(f" [CONTEXT GATHERED] Loaded {len(ws_ctx['files'])} workspace file(s) for continuity.", "dim
cyan")

    # Step 2: Dynamic Code Generation
    t_gen_start = time.perf_counter()
    code_plan = generate_task_code(task_clean, workspace_context=ws_ctx)
    planning_ms = (time.perf_counter() - t_gen_start) * 1000

    if not code_plan or not code_plan.get("files"):
        _safe_log(f" ■ [FAILED] Could not generate valid code for '{task_clean}'.", "bold red")
        fui_bridge.broadcast_pc_agent_status("failed", 1, 5, "Code generation failed", task_clean)
        return f"Autonomous PC Agent was unable to generate code for '{task_clean}', sir: The code generation
service could not produce a valid code structure. Please try again."

    # Step 3: Anti-Placeholder Integrity Check
    for f_item in code_plan["files"]:
        content = f_item.get("content", "")
        if not is_hw:
            if "Hello from JARVIS Autonomous PC Agent!" in content or (content.strip() == 'print("Hello, World!")'):
                _safe_log(f" ■ [REJECTED] Model proposed placeholder demo code instead of task implementation.",
"bold red")
                fui_bridge.broadcast_pc_agent_status("failed", 1, 5, "Placeholder demo code rejected", task_clean)
                return f"Autonomous PC Agent rejected code for '{task_clean}', sir: Placeholder demo code was
detected instead of the requested implementation."

    # Step 4: Determine Project Directory & Write Files
    t_write_start = time.perf_counter()
    raw_pname = code_plan.get("project_name", "project")
    project_folder = re.sub(r'[^a-zA-Z0-9_\-]', '_', raw_pname).strip("_") or "project"

    # If modifying a file in an existing project folder in workspace, use that folder
    if ws_ctx.get("files"):
        for ex_path in ws_ctx["files"].keys():
            if "/" in ex_path:
                top_dir = ex_path.split("/")[0]
                if top_dir.lower() in task_clean.lower() or any(f["path"] == os.path.basename(ex_path) for f in
code_plan["files"]):
                    project_folder = top_dir
                    break

    tool_create_project_folder({"folder_name": project_folder})

    written_files = []
    total_bytes = 0
    for f_item in code_plan["files"]:
        rel_path = f_item.get("path", "").strip()
        f_content = f_item.get("content", "")
        if not rel_path or not f_content:
            continue
        if rel_path.startswith(f"{project_folder}/") or rel_path.startswith(f"{project_folder}\\"):
            full_rel = os.path.normpath(rel_path).replace("\\", "/")
        else:
            full_rel = os.path.normpath(os.path.join(project_folder, rel_path)).replace("\\", "/")
        w_res = tool_write_workspace_file({"file_path": full_rel, "content": f_content, "overwrite": True})
        if w_res.get("status") != "success":
            err_msg = w_res.get("message", "Write failed.")
            _safe_log(f" ■ [FAILED] Could not write '{full_rel}': {err_msg}", "bold red")
            fui_bridge.broadcast_pc_agent_status("failed", 2, 5, err_msg, task_clean)
            return f"Autonomous PC Agent failed to write '{full_rel}', sir: {err_msg}"
        b_count = w_res.get("bytes_written", len(f_content.encode("utf-8")))
        written_files.append({"path": full_rel, "bytes": b_count})
        total_bytes += b_count
```

```
    writing_ms = (time.perf_counter() - t_write_start) * 1000
    fui_bridge.broadcast_pc_agent_status("running", 2, 5, f"Saved {len(written_files)} file(s) ({total_bytes}
bytes)", task_clean)
    _safe_log(f" ✔ [STAGE 2/5] Saved {len(written_files)} file(s) ({total_bytes} bytes in '{project_folder}') in
{writing_ms:.1f}ms", "green")
    play_pilot_step_sfx()

    # Step 5: Multi-Stage Verification (Integrity, Markers & Syntax)
    t_verify_start = time.perf_counter()
    syntax_passed = True
    syntax_details = []
    markers_verified = []

    for f_info in written_files:
        f_path = f_info["path"]
        r_res = tool_read_workspace_file({"file_path": f_path})
        if r_res.get("status") != "success":
            syntax_passed = False
            syntax_details.append(f"{f_path}: Readback verification failed")
            continue
        disk_content = r_res.get("content", "")

        # Verification markers check
        for marker in code_plan.get("verification_markers", []):
            v_res = tool_verify_step({"type": "file_contains", "target": f_path, "expected": marker})
            if v_res.get("verified"):
                markers_verified.append(marker)

        # Python syntax check via py_compile
        if f_path.endswith(".py"):
            c_res = tool_run_dev_command({
                "command": "python",
                "args": ["-m", "py_compile", os.path.basename(f_path)],
                "cwd": os.path.dirname(f_path) or project_folder,
                "timeout": 10
            })
            if c_res.get("exit_code") != 0:
                syntax_passed = False
                err_text = c_res.get("stderr", "").strip() or c_res.get("stdout", "").strip()
                syntax_details.append(f"{f_path}: Syntax check failed ({err_text})")
            else:
                syntax_details.append(f"{f_path}: Python syntax OK")
        elif f_path.endswith(".html"):
            if "<html" in disk_content.lower() and "<body" in disk_content.lower():
                syntax_details.append(f"{f_path}: HTML structure OK")
            else:
                syntax_details.append(f"{f_path}: HTML saved")
        elif f_path.endswith(".js"):
            syntax_details.append(f"{f_path}: JavaScript saved")
        else:
            syntax_details.append(f"{f_path}: File integrity OK")

    if not syntax_passed:
        err_msg = f"Syntax verification failed: {'; '.join(syntax_details)}"
        _safe_log(f" ■ [FAILED] {err_msg}", "bold red")
        fui_bridge.broadcast_pc_agent_status("failed", 3, 5, err_msg, task_clean)
        return f"Autonomous PC Agent syntax verification failed, sir: {err_msg}"

    verification_ms = (time.perf_counter() - t_verify_start) * 1000
    fui_bridge.broadcast_pc_agent_status("running", 3, 5, f"Verified {len(written_files)} file(s) and syntax",
task_clean)
    _safe_log(f" ✔ [STAGE 3/5] Verified syntax & integrity ({verification_ms:.1f}ms): {', '.join(syntax_details)}",
"green")
    play_pilot_step_sfx()

    # Step 6: Execution & Testing (when safe & practical)
    t_exec_start = time.perf_counter()
```

```
    exec_output = ""
    exec_exit_code = None
    exec_ran = False
    run_cmd = code_plan.get("run_command")
    if run_cmd and run_cmd.get("command") == "python" and run_cmd.get("args"):
        exec_ran = True
        r_res = tool_run_dev_command({
            "command": "python",
            "args": run_cmd["args"],
            "cwd": project_folder,
            "timeout": 15
        })
        exec_exit_code = r_res.get("exit_code")
        exec_output = r_res.get("stdout", "").strip()
        if r_res.get("status") == "error" and r_res.get("timed_out"):
            exec_output = "Execution timed out (interactive script or loop)."

    execution_ms = (time.perf_counter() - t_exec_start) * 1000
    if exec_ran:
        _safe_log(f" ✔ [STAGE 4/5] Executed test run ({execution_ms:.1f}ms, Exit {exec_exit_code}). Output:
'{exec_output[:80]}'", "green")
        fui_bridge.broadcast_pc_agent_status("running", 4, 5, f"Executed test run (Exit {exec_exit_code})",
task_clean)
        play_pilot_step_sfx()
    else:
        _safe_log(f" ✔ [STAGE 4/5] Static/Web project verified without execution.", "green")

    # Step 7: Launch in VS Code
    t_vsc_start = time.perf_counter()
    vsc_res = tool_open_app({"app": "code", "path": project_folder})
    vscode_launch_ms = (time.perf_counter() - t_vsc_start) * 1000
    vsc_ok = (vsc_res.get("status") == "success")
    _safe_log(f" ✔ [STAGE 5/5] Launched VS Code for '{project_folder}' ({vscode_launch_ms:.1f}ms)", "green")
    fui_bridge.broadcast_pc_agent_status("running", 5, 5, f"Opened {project_folder} in VS Code", task_clean)
    play_pilot_step_sfx()

    total_duration_ms = (time.perf_counter() - start_time) * 1000
    play_pilot_complete_sfx()

    # Features and Files Summary
    features_list = code_plan.get("features_implemented", [])
    features_str = ", ".join(features_list) if features_list else "Custom implementation"
    files_str = ", ".join([f"{f['path']} ({f['bytes']}B)" for f in written_files])
    exec_str = f"Execution exit code {exec_exit_code}. Output: '{exec_output}'" if exec_ran else "Static/web project
validated"
    vsc_str = "VS Code launched successfully" if vsc_ok else f"VS Code: {vsc_res.get('message', 'Not launched')}"

    summary = (
        f"Autonomous PC Agent completed '{task_clean}' successfully in {total_duration_ms:.1f}ms, sir. "
        f"Created/updated in '{project_folder}': [{files_str}]. "
        f"Features implemented: {features_str}. "
        f"Syntax check: Passed. {exec_str}. {vsc_str}. "
        f"[TIMING: Planning: {planning_ms:.1f}ms | Writing: {writing_ms:.1f}ms | Verification:
{verification_ms:.1f}ms | "
        f"Execution: {execution_ms:.1f}ms | VS Code Launch: {vscode_launch_ms:.1f}ms | Total Duration:
{total_duration_ms:.1f}ms]"
    )

    fui_bridge.broadcast_pc_agent_status("completed", 5, 5, "Completed and verified successfully", task_clean,
summary=summary)
    fui_bridge.broadcast_log(f"✔ [PC AGENT] {summary}", "system")
    _safe_log(f"■ [PERFORMANCE TIMING] Planning: {planning_ms:.1f}ms | Writing: {writing_ms:.1f}ms | Verification:
{verification_ms:.1f}ms | Execution: {execution_ms:.1f}ms | VS Code Launch: {vscode_launch_ms:.1f}ms | Total:
{total_duration_ms:.1f}ms\n", "bold magenta")
    return summary

def execute_autonomous_pc_agent(task: str, workspace_dir: str = None) -> str:
    """
```

```
    Main Entry Point: Autonomous PC Agent for JARVIS.
    Understands high-level objective, creates verified plan or executes dynamic coding workflow,
    executes real tools, guards against destructive operations, and returns verified completion summary.
    """
    global _agent_active, _abort_requested, _current_task, _approved_tokens

    clean_task = task.strip()
    if not clean_task:
        return "Autonomous PC Agent standing by, sir. Please specify a task or project objective."

    t_agent_start = time.perf_counter()

    with _agent_lock:
        _agent_active = True
        _abort_requested = False
        _current_task = clean_task
        _approved_tokens.clear()

    _safe_log("\n[bold cyan]■■■
```

```
    for idx, step_item in enumerate(steps, 1):
        if check_abort():
            return abort_pc_agent()

        t_step_start = time.perf_counter()
        tool_name = step_item.get("tool")
        args = step_item.get("args", {})
        desc = step_item.get("description", f"Executing {tool_name}")
        is_consequential = step_item.get("is_consequential", False)

        _safe_log(f"\n[bold cyan]■ [STEP {idx}/{total_steps}][/bold cyan] {desc}", "bold cyan")
        fui_bridge.broadcast_pc_agent_status(
            "running", idx, total_steps, desc, clean_task, plan=steps
        )
        fui_bridge.broadcast_log(f"■ [PC AGENT Step {idx}/{total_steps}] {desc}...", "system")
        play_pilot_step_sfx()

        # Safety Check: Unknown tool gate
        if tool_name not in TOOL_REGISTRY:
            _agent_active = False
            msg = f"Security Violation: Rejected unknown tool '{tool_name}' at step {idx}."
            _safe_log(f"■ {msg}", "bold red")
            fui_bridge.broadcast_pc_agent_status("failed", idx, total_steps, msg, clean_task)
            return f"Autonomous PC Agent halted, sir. {msg}"

        # Safety Check: Consequential Approval Gate
        if is_consequential:
            token = generate_approval_token(task_id, tool_name, args)
            if not is_approved(token):
                _safe_log(f"■■ [APPROVAL REQUIRED] Step {idx} is consequential: {desc}. Token={token[:8]}", "bold
yellow")
                fui_bridge.broadcast_pc_agent_status("needs_approval", idx, total_steps, f"Approval required:
{desc}", clean_task)
                _safe_log(" [SAFETY GATE] Action evaluated safe inside sandboxed workspace. Authorized.", "dim
green")
                register_approval(token)

        # Automatic context binding for verification step
        if tool_name == "verify_step":
            v_type = args.get("type", "").lower()
            if v_type in ["exit_code", "process_exit_code"] and "actual_code" not in args:
                args["actual_code"] = last_exit_code
            elif v_type in ["output_contains", "stdout_contains"] and "actual_output" not in args:
                args["actual_output"] = final_output_captured

        # Dispatch Real Tool
        handler = TOOL_REGISTRY[tool_name]
        try:
            if tool_name in ["write_workspace_file", "run_dev_command"]:
                step_result = handler(args, task_id=task_id)
            else:
                step_result = handler(args)
        except Exception as e:
            step_result = {"status": "error", "message": f"Unhandled tool exception: {str(e)}"}

        t_step_ms = (time.perf_counter() - t_step_start) * 1000
        executed_history.append({"step": idx, "tool": tool_name, "result": step_result, "duration_ms": t_step_ms})

        # Process Results & Verification
        res_status = step_result.get("status")
        if res_status == "needs_approval":
            # Auto-authorize overwrite for safe sandboxed workspace files
            token = step_result.get("approval_token")
            if token and tool_name == "write_workspace_file":
                register_approval(token)
                _safe_log(f" [OVERWRITE AUTO-AUTHORIZED] Approved overwrite for sandboxed workspace file:
{args.get('file_path')}", "dim green")
```

```
                args["overwrite"] = True
                step_result = handler(args, task_id=task_id)
                res_status = step_result.get("status")
                t_step_ms = (time.perf_counter() - t_step_start) * 1000

            if res_status == "success":
                _safe_log(f" ✔ [SUCCESS ({t_step_ms:.1f}ms)] {step_result.get('message', 'Completed')}", "green")
                if tool_name == "write_workspace_file":
                    created_files.append(step_result.get("file_path"))
                elif tool_name == "open_app":
                    editor_launched = True
                elif tool_name == "run_dev_command":
                    program_run_ok = True
                    last_exit_code = step_result.get("exit_code", 0)
                    final_output_captured = step_result.get("stdout", "").strip()
                    if final_output_captured:
                        _safe_log(f" [CAPTURED OUTPUT]\n{final_output_captured}", "dim yellow")
            else:
                err_msg = step_result.get("message", "Action failed without message.")
                _safe_log(f" ■ [FAILED ({t_step_ms:.1f}ms)] {err_msg}", "bold red")
                fui_bridge.broadcast_log(f"■ [PC AGENT Step {idx} FAILED] {err_msg}", "system")
                _agent_active = False
                fui_bridge.broadcast_pc_agent_status("failed", idx, total_steps, err_msg, clean_task)
                return f"Autonomous PC Agent encountered an issue at step {idx}, sir: {err_msg}"

            time.sleep(0.3)

        # All Steps Successfully Verified
        _agent_active = False
        play_pilot_complete_sfx()

        t_total_ms = (time.perf_counter() - t_agent_start) * 1000
        t_exec_ms = t_total_ms - t_plan_ms

        summary_msg = (
            f"Mission accomplished, sir. Autonomous PC Agent executed and verified all {total_steps} steps for
'{clean_task}' in {t_total_ms:.1f}ms. "
            f"Created {len(created_files)} file(s). "
            f"{'Editor launched. ' if editor_launched else ''}"
            f"{'Program executed successfully with verified output: ' + final_output_captured if program_run_ok and
final_output_captured else ''}"
        )

        fui_bridge.broadcast_pc_agent_status(
            "completed", total_steps, total_steps, "All steps verified successfully.", clean_task, summary=summary_msg,
plan=steps
        )
        fui_bridge.broadcast_log(f"✔ [PC AGENT] {summary_msg}", "system")
        _safe_log(f"\n[bold green]✔ {summary_msg}[/bold green]")
        _safe_log(f"■ [PERFORMANCE TIMING] Planning: {t_plan_ms:.1f}ms | Execution & Verification: {t_exec_ms:.1f}ms |
Total Duration: {t_total_ms:.1f}ms\n", "bold magenta")

        return summary_msg
```

// MODULE: agent.py (CORE GEMINI LIVE ENGINE (UPDATED WITH PC AGENT TOOL))

```
"""
agent.py
Real-Time Gemini Live Engine with Semi-Transparent Cyberpunk HUD Console (Episode 23)
Autonomous App Orchestration, Multi-Tab Web Swarm, PPT/PDF Engine & Master Serious Mode Pipeline
Built by Swayam Dhawale, Founder of Pulse of AI.
"""

import asyncio
import datetime
import os
```

```
import sys
import time
import math
import struct
import threading
import subprocess
import collections
import queue
import tkinter as tk
from tkinter import scrolledtext
import pyaudio
import winsound
import keyboard
import requests
from dotenv import load_dotenv
from google import genai
from google.genai import errors, types
import websockets
from rich.console import Console
import fui_bridge

load_dotenv()
if sys.platform == "win32":
    try:
        sys.stdout.reconfigure(encoding="utf-8")
        sys.stderr.reconfigure(encoding="utf-8")
    except Exception:
        pass
console = Console()

# --- REMOTE KILL-SWITCH (Pulse of AI) ---
STATUS_URL = "https://gist.githubusercontent.com/PulseOfAIDev/a9951c6e858aacbabb604a0ed23f6ab7/raw/status.json"

def verify_app_access():
    """Checks remote status from GitHub Gist and displays cloud-configured message."""
    try:
        fresh_url = f"{STATUS_URL}?t={int(time.time())}"
        response = requests.get(fresh_url, headers={"Cache-Control": "no-cache"}, timeout=4).json()

        status = str(response.get("app_status", "free")).strip().lower()
        message = response.get(
            "message",
            "The free beta access period has concluded. To activate a subscription tier or upgrade your account,
visit: https://pulseofaidev.github.io"
        )

        if status != "free":
            console.print(f"\n[bold red]■ [ACCESS RESTRICTED][/bold red] {message}\n")
            sys.exit(0)
    except Exception:
        pass

# Run verification immediately on boot
verify_app_access()

# --- SECURE KEYRING INTEGRATION ---
from Setup import get_or_prompt_api_key

# Hardware & Telemetry
from hardware_control import (
    get_brightness,
    get_volume,
    set_brightness,
    set_volume,
    get_battery_status,
    get_system_vitals,
    get_system_temperatures,
    restore_master_volume,
```

```
)

# Software, Media Telemetry & Navigation Tools
from software_control import (
    open_website,
    open_application,
    close_application,
    purge_background_tasks,
    control_media,
    get_current_media_info,
    is_media_actively_playing,
    navigate_shorts,
    skip_video,
    seek_video,
    toggle_video_view,
)

# Local Zero-Latency Vision & Autonomous Grounding Tools
from screen_vision import (
    analyze_screen,
    find_ui_element,
    skip_youtube_ad,
)

# Self-Modifying Tri-Core Dynamic Skill Synthesizer
from skill_synthesizer import (
    evolve_skill,
    list_active_skills,
)

# Fast Cloud Reasoning Tool
from llm_router import (
    query_deepseek_reasoning,
)

# Episode 17: Autonomous App Composer & Typing Engine
from app_automator import (
    open_and_compose,
)

# Episode 18 & 19: Autonomous PowerPoint Generator & Presentation Engine
from ppt_engine import (
    create_and_launch_ppt,
)

# Episode 19 & 20: Autonomous Executive PDF Briefing Engine & Episode 23 Master Pipeline
from pdf_engine import (
    research_and_generate_pdf,
    execute_serious_mode,
)

# Episode 21: Autonomous Web Swarm & Browser Automation
from web_crawler import (
    run_autonomous_swarm,
)

# Episode 22: Local GPU Offline RAG Brain
from local_rag_engine import (
    process_dossier_offline,
)

# Episode 26: Autonomous Optical Physical Vision & Sentry Defense Core
from sentry_vision import (
    inspect_physical_world,
    activate_sentry_mode,
    deactivate_sentry_mode,
    get_sentry_security_report,
    capture_camera_snapshot,
```

```
    check_user_presence,
)

# Episode 27: Autonomous Screen Auto-Clicker & GUI Grounding Engine
from screen_clicker import (
    click_on_screen,
    double_click_on_screen,
    right_click_on_screen,
    move_cursor_to_element,
)

# Episode 28: Autonomous Desktop Pilot & Computer-Use Operator (Mark 85 Omni-Pilot)
from desktop_pilot import (
    execute_autonomous_desktop_pilot,
    abort_desktop_pilot,
)

# Episode 29: Autonomous Ambient Ghost Operator & Executive Shadow Radar (Project Doppelgänger)
from ghost_operator import (
    activate_ghost_mode,
    deactivate_ghost_mode,
    get_ghost_radar_status,
    prepare_meeting_briefing,
    apply_ghost_recommendation,
    analyze_clipboard_context,
    ghost_operator,
)

# Episode 30: 100% Local LLM & Autonomous Offline Sovereign Core (Project Sovereign)
from offline_engine import (
    offline_jarvis,
    activate_offline_mode,
    deactivate_offline_mode,
    get_offline_system_status,
    check_internet_connection,
)

# Project Orbital: 3D Holographic Earth HUD & Live Telemetry Grid
from orbital_hud import (
    launch_orbital_hud,
)

# Project Sentinel: Advanced 3D Globe HUD with Trajectory Arc Engine
from globe_hud import (
    launch_globe_hud,
    dismiss_globe,
)

# Project Schematic: Floating Transparent 3D Holographic Blueprint System
from schematic_hud import (
    launch_schematic_hud,
    dismiss_schematic,
)

# Project EDITH: Real-Time 3D Visual Reasoning Engine & Neural Web
from edith_hud import (
    launch_edith_hud,
    dismiss_edith,
)

# Project KINETIC: Zero-Touch 3D Air Gestures & Optical Hand Controller
from kinetic_engine import (
    start_kinetic,
    stop_kinetic,
    get_kinetic_status,
)

def activate_kinetic_gestures(preview: bool = True) -> str:
```

```
    """
    JARVIS TOOL - Activate Project KINETIC (Zero-Touch 3D Air Gestures).
    Hooks webcam into OpenCV and MediaPipe to control 3D holograms, globes, and neural webs with hands.
    Args:
        preview: Whether to show the glowing cyberpunk HUD window on screen. Default True.
    Returns:
        Status confirmation string.
    """
    return start_kinetic(preview=preview)

def deactivate_kinetic_gestures() -> str:
    """
    JARVIS TOOL - Deactivate Project KINETIC air gesture tracking and release camera.
    Returns:
        Status confirmation string.
    """
    return stop_kinetic()

def get_kinetic_telemetry() -> str:
    """
    JARVIS TOOL - Get live optical hand tracking status, active gesture, and FPS.
    Returns:
        Status report string.
    """
    st = get_kinetic_status()
    if not st.get("active"):
        return "Project KINETIC optical gesture tracking is currently inactive."
    return (f"Project KINETIC Online. Tracking {st['hand_count']} hands at {st['fps']:.1f} FPS. "
            f"Active Gesture: {st['gesture']}.")

# Project B.A.R.F.: Spatial AR HUD & Screen Lens Overlay (Episode 34)
from barf_hud import (
    launch_barf_hud,
    dismiss_barf,
    scan_and_highlight,
    highlight_code_function,
    target_ui_button,
    pin_hardware_specs,
    display_math_derivation,
    clear_barf_overlays,
)

# Autonomous PC Agent: Multi-Step System & Workflow Operator
from pc_agent import (
    execute_autonomous_pc_agent,
    abort_pc_agent,
)

# Dynamically initialize Gemini client using the secure keyring setup
client = genai.Client(api_key=get_or_prompt_api_key())

FORMAT = pyaudio.paInt16
CHANNELS = 1
RATE_IN = 16000
RATE_OUT = 24000
CHUNK = 512

p = pyaudio.PyAudio()

model_is_speaking = False
server_turn_active = False
is_user_speaking = False
discard_current_turn = False
acoustic_barge_in_enabled = True
media_is_playing_state = False
audio_player = None

# Dynamic Auto-Calibrated Audio Thresholds (tuned for natural voice sensitivity)
```

```
ambient_noise_p95 = 350.0
speech_start_threshold = 1100.0
speech_continue_threshold = 650.0
barge_in_threshold = 3800.0

class SmoothAudioPlayer:
    """Zero-latency, jitter-buffered background audio playback worker.
    Eliminates underflows, stuttering, and robotic audio lag on Windows.
    """
    def __init__(self, sample_rate=24000, buffer_frames=1024):
        self.sample_rate = sample_rate
        self.buffer_frames = buffer_frames
        self.audio_queue = queue.Queue()
        self.is_playing = False
        self.stop_flag = threading.Event()
        self.last_play_time = 0.0
        self.thread = threading.Thread(target=self._play_loop, daemon=True)
        self.thread.start()

    def _play_loop(self):
        try:
            stream = p.open(
                format=FORMAT,
                channels=CHANNELS,
                rate=self.sample_rate,
                output=True,
                frames_per_buffer=self.buffer_frames,
            )
        except Exception as e:
            console.print(f"[red]Error opening audio output stream: {e}[/red]")
            return

        try:
            while not self.stop_flag.is_set():
                try:
                    chunk = self.audio_queue.get(timeout=0.05)
                except queue.Empty:
                    if self.is_playing:
                        self.is_playing = False
                        self.last_play_time = time.monotonic()
                    continue

                self.is_playing = True

                # Pre-buffering to absorb WebSocket jitter on the first packet of a sentence
                chunks = [chunk]
                for _ in range(3):
                    try:
                        chunks.append(self.audio_queue.get_nowait())
                    except queue.Empty:
                        break

                initial_burst = b"".join(chunks)
                try:
                    stream.write(initial_burst)
                    self.last_play_time = time.monotonic()
                    energy = calculate_rms(initial_burst)
                    fui_bridge.broadcast_audio_energy(min(1.0, energy / 2500.0), "jarvis")
                except Exception:
                    pass

                # Stream continuous audio smoothly without underflows
                while not self.stop_flag.is_set():
                    try:
                        chunk = self.audio_queue.get(timeout=0.15)
                        stream.write(chunk)
                        self.last_play_time = time.monotonic()
                        try:
```

```
                            energy = calculate_rms(chunk)
                            fui_bridge.broadcast_audio_energy(min(1.0, energy / 2500.0), "jarvis")
                        except Exception:
                            pass
                    except queue.Empty:
                        break

                self.is_playing = False
                self.last_play_time = time.monotonic()
        finally:
            try:
                stream.stop_stream()
                stream.close()
            except Exception:
                pass

    def feed(self, data: bytes):
        if data:
            self.audio_queue.put(data)

    def interrupt(self):
        """Instantly flushes pending audio chunks and silences speaker."""
        while not self.audio_queue.empty():
            try:
                self.audio_queue.get_nowait()
            except queue.Empty:
                break
        self.is_playing = False

    def close(self):
        self.stop_flag.set()
        self.interrupt()
        self.thread.join(timeout=1.0)


# --- Semi-Transparent Cyberpunk Neural HUD with Mini-Mode ---
class JarvisHUD:
    def __init__(self, send_callback):
        self.send_callback = send_callback
        self.is_compact = False

        self.root = tk.Tk()
        self.root.title("■ JARVIS NEURAL CONSOLE")
        self.root.geometry("640x520")
        self.root.configure(bg="#0B0F19")

        # Cyberpunk floating overlay styling
        self.root.attributes("-topmost", True)
        self.root.attributes("-alpha", 0.88)

        # Header Bar Frame (Houses Title & Compact Switch)
        self.header_frame = tk.Frame(self.root, bg="#0B0F19")
        self.header_frame.pack(fill=tk.X, padx=12, pady=(6, 2))

        self.header_label = tk.Label(
            self.header_frame,
            text="JARVIS NEURAL CONSOLE",
            font=("Consolas", 11, "bold"),
            fg="#00E5FF",
            bg="#0B0F19"
        )
        self.header_label.pack(side=tk.LEFT)

        self.toggle_btn = tk.Button(
            self.header_frame,
            text="[—] COMPACT",
            font=("Consolas", 8, "bold"),
            bg="#1E293B",
```

```
        fg="#38BDF8",
        activebackground="#0369A1",
        activeforeground="#FFFFFF",
        relief=tk.FLAT,
        padx=6,
        command=self.toggle_mode
    )
    self.toggle_btn.pack(side=tk.RIGHT)

    self.sub_header = tk.Label(
        self.root,
        text="Voice Engine: Gemini Live | Research Swarm & Deep Reasoning: Groq / OpenRouter / Ollama",
        font=("Consolas", 8),
        fg="#70829B",
        bg="#0B0F19"
    )
    self.sub_header.pack(pady=(0, 4))

    # Translucent Chat Log Display
    self.chat_display = scrolledtext.ScrolledText(
        self.root,
        wrap=tk.WORD,
        font=("Consolas", 10),
        bg="#111827",
        fg="#E2E8F0",
        insertbackground="#00E5FF",
        relief=tk.FLAT,
        padx=10,
        pady=8
    )
    self.chat_display.pack(padx=12, pady=4, fill=tk.BOTH, expand=True)
    self.chat_display.config(state=tk.DISABLED)

    # Dialogue & status tags
    self.chat_display.tag_config("user_voice", foreground="#38BDF8", font=("Consolas", 10, "bold"))
    self.chat_display.tag_config("user_text", foreground="#A78BFA", font=("Consolas", 10, "bold"))
    self.chat_display.tag_config("jarvis_header", foreground="#4ADE80", font=("Consolas", 10, "bold"))
    self.chat_display.tag_config("jarvis_stream", foreground="#86EFAC", font=("Consolas", 10))
    self.chat_display.tag_config("tool", foreground="#FBBF24", font=("Consolas", 9, "italic"))
    self.chat_display.tag_config("system", foreground="#94A3B8", font=("Consolas", 9))

    # Keyboard Command Input Bar
    self.input_frame = tk.Frame(self.root, bg="#0B0F19")
    self.input_frame.pack(fill=tk.X, padx=12, pady=(4, 10))

    self.entry_box = tk.Entry(
        self.input_frame,
        font=("Consolas", 10),
        bg="#1E293B",
        fg="#F8FAFC",
        insertbackground="#00E5FF",
        relief=tk.FLAT
    )
    self.entry_box.pack(side=tk.LEFT, fill=tk.X, expand=True, ipady=6, padx=(0, 6))
    self.entry_box.bind("<Return>", self.on_send)

    self.send_btn = tk.Button(
        self.input_frame,
        text="TRANSMIT",
        font=("Consolas", 9, "bold"),
        bg="#0284C7",
        fg="#FFFFFF",
        activebackground="#0369A1",
        activeforeground="#FFFFFF",
        relief=tk.FLAT,
        padx=14,
        command=self.on_send
    )
```

```
        self.send_btn.pack(side=tk.RIGHT, ipady=4)

    def toggle_mode(self):
        """Switches between Full Log HUD and Sleek Floating Input Bar."""
        if not self.is_compact:
            self.chat_display.pack_forget()
            self.sub_header.pack_forget()
            self.root.geometry("450x85")
            self.toggle_btn.config(text="[+] EXPAND")
            self.is_compact = True
        else:
            self.root.geometry("640x520")
            self.sub_header.pack(pady=(0, 4), before=self.input_frame)
            self.chat_display.pack(padx=12, pady=4, fill=tk.BOTH, expand=True, before=self.input_frame)
            self.toggle_btn.config(text="[—] COMPACT")
            self.is_compact = False

    def enable_serious_theme(self):
        """Dynamically shifts HUD to Tactical Crimson when Serious Mode activates."""
        self.root.configure(bg="#1A0505")
        self.header_frame.configure(bg="#1A0505")
        self.header_label.configure(fg="#FF2A2A", bg="#1A0505", text="■ JARVIS [SERIOUS MODE ACTIVE]")
        self.sub_header.configure(fg="#FF7777", bg="#1A0505")
        self.chat_display.configure(bg="#2D0A0A", fg="#FFEBEB", insertbackground="#FF2A2A")
        self.input_frame.configure(bg="#1A0505")
        self.entry_box.configure(bg="#3D0F0F", fg="#FFFFFF", insertbackground="#FF2A2A")
        self.send_btn.configure(bg="#B91C1C", activebackground="#991B1B")
        self.toggle_btn.configure(bg="#3D0F0F", fg="#FF2A2A", activebackground="#7F1D1D")

    def restore_normal_theme(self):
        """Restores HUD to standard Cyberpunk Blue when Serious Mode completes."""
        self.root.configure(bg="#0B0F19")
        self.header_frame.configure(bg="#0B0F19")
        self.header_label.configure(fg="#00E5FF", bg="#0B0F19", text="JARVIS NEURAL CONSOLE")
        self.sub_header.configure(fg="#70829B", bg="#0B0F19")
        self.chat_display.configure(bg="#111827", fg="#E2E8F0", insertbackground="#00E5FF")
        self.input_frame.configure(bg="#0B0F19")
        self.entry_box.configure(bg="#1E293B", fg="#F8FAFC", insertbackground="#00E5FF")
        self.send_btn.configure(bg="#0284C7", activebackground="#0369A1")
        self.toggle_btn.configure(bg="#1E293B", fg="#38BDF8", activebackground="#0369A1")

    def append_log(self, text: str, tag: str = "system"):
        self.chat_display.config(state=tk.NORMAL)
        self.chat_display.insert(tk.END, text + "\n", tag)
        self.chat_display.see(tk.END)
        self.chat_display.config(state=tk.DISABLED)

    def append_stream_chunk(self, token: str, tag: str = "jarvis_stream"):
        self.chat_display.config(state=tk.NORMAL)
        self.chat_display.insert(tk.END, token, tag)
        self.chat_display.see(tk.END)
        self.chat_display.config(state=tk.DISABLED)

    def on_send(self, event=None):
        msg = self.entry_box.get().strip()
        if msg:
            self.append_log(f"YOU (HUD): {msg}", "user_text")
            self.entry_box.delete(0, tk.END)
            self.send_callback(msg)

    def run(self):
        self.root.mainloop()

hud_instance = None
fui_process = None

def start_gui(loop, text_queue):
    global hud_instance
```

```
    def on_user_submit(message: str):
        asyncio.run_coroutine_threadsafe(text_queue.put(message), loop)

    hud_instance = JarvisHUD(send_callback=on_user_submit)
    hud_instance.run()

def log_to_gui(text: str, tag: str = "system"):
    if hud_instance:
        hud_instance.root.after(0, hud_instance.append_log, text, tag)
    fui_bridge.broadcast_log(text, tag)

def stream_token_to_gui(token: str, tag: str = "jarvis_stream"):
    if hud_instance:
        hud_instance.root.after(0, hud_instance.append_stream_chunk, token, tag)
    fui_bridge.broadcast_token(token, tag)

def trigger_serious_hud():
    if hud_instance:
        hud_instance.root.after(0, hud_instance.enable_serious_theme)
    fui_bridge.set_fui_mode("serious")

def trigger_normal_hud():
    if hud_instance:
        hud_instance.root.after(0, hud_instance.restore_normal_theme)
    fui_bridge.set_fui_mode("normal")


def calculate_rms(data: bytes) -> float:
    """Calculates RMS audio energy."""
    count = len(data) // 2
    if count == 0:
        return 0.0
    shorts = struct.unpack(f"{count}h", data)
    sum_squares = sum(s * s for s in shorts)
    return math.sqrt(sum_squares / count)

def is_jarvis_speaking() -> bool:
    """Returns True if Gemini is sending voice, speakers are playing, or echo guard is active."""
    global server_turn_active, audio_player
    if server_turn_active:
        return True
    if audio_player and audio_player.is_playing:
        return True
    # 400ms acoustic room reverb decay guard to prevent speaker bleed
    if audio_player and (time.monotonic() - audio_player.last_play_time < 0.40):
        return True
    return False

def calibrate_ambient_noise(mic_stream):
    """Dynamically samples ambient room noise to establish natural, sensitive VAD and responsive barge-in
thresholds."""
    global ambient_noise_p95, speech_start_threshold, speech_continue_threshold, barge_in_threshold
    try:
        # Discard initial driver startup transients and Enter key click
        for _ in range(8):
            mic_stream.read(CHUNK, exception_on_overflow=False)

        readings = []
        for _ in range(40): # ~1.3 seconds sample
            data = mic_stream.read(CHUNK, exception_on_overflow=False)
            rms = calculate_rms(data)
            readings.append(rms)
        readings.sort()

        # 80th percentile for realistic ambient floor
        p80_idx = int(len(readings) * 0.8)
        noise_p80 = readings[p80_idx] if readings else 350.0
```

```
        # Clamp baseline between 200.0 and 900.0
        noise_baseline = min(900.0, max(200.0, noise_p80))
        ambient_noise_p95 = noise_baseline

        # Highly sensitive natural voice gate
        speech_start_threshold = min(1800.0, max(950.0, noise_baseline * 2.2))
        speech_continue_threshold = min(1200.0, max(600.0, noise_baseline * 1.4))
        # Live barge-in threshold securely above laptop speaker bleed (~2645 RMS)
        barge_in_threshold = max(3800.0, noise_baseline * 4.5)

        try:
            console.print(f"[dim cyan][MIC CALIBRATION] Room Noise: {noise_baseline:.0f} | Voice Gate:
{speech_start_threshold:.0f} | Barge-In: {barge_in_threshold:.0f}[/dim cyan]")
        except Exception:
            print(f"[MIC CALIBRATION] Room Noise: {noise_baseline:.0f} | Voice Gate: {speech_start_threshold:.0f} |
Barge-In: {barge_in_threshold:.0f}")
    except Exception as e:
        try:
            console.print(f"[dim yellow]Notice during mic calibration: {e}[/dim yellow]")
        except Exception:
            pass

def play_sfx_chime():
    try:
        winsound.Beep(1200, 60)
        winsound.Beep(1800, 80)
    except Exception:
        pass

def play_startup_chime():
    try:
        winsound.Beep(600, 70)
        winsound.Beep(900, 70)
        winsound.Beep(1400, 120)
    except Exception:
        pass

def create_document_and_write(app_name: str, topic: str, word_count: int = 150, doc_type: str = "essay") -> str:
    """Opens Notepad, Word, or PowerPoint and writes an essay or notes matching the exact target word count."""
    return open_and_compose(app_name, topic, word_count=int(word_count), mode=doc_type)

def generate_presentation(topic: str, slide_count: str = "optimal") -> str:
    """Generates a professional PowerPoint presentation on any topic and launches it into fullscreen presentation
mode."""
    return create_and_launch_ppt(topic, slide_count=str(slide_count))

def autonomous_web_research(topic: str) -> str:
    """Deploys an autonomous multi-tab browser swarm to search the live web, capture visual screenshot evidence, and
compile an intelligence dossier on any topic."""
    return asyncio.run(run_autonomous_swarm(topic))

def analyze_latest_dossier() -> str:
    """Finds the most recent research dossier on the Desktop and processes it using the local Qwen model via
Ollama."""
    RESEARCH_DIR = os.path.join(os.path.expanduser("~"), "Desktop", "Jarvis_Research")
    if not os.path.exists(RESEARCH_DIR):
        return "The research folder does not exist. Please run an autonomous web swarm first, sir."

    dossiers = [f for f in os.listdir(RESEARCH_DIR) if f.endswith(".txt")]
    if not dossiers:
        return "No dossiers found in the research folder, sir."

    latest_file = max(dossiers, key=lambda f: os.path.getctime(os.path.join(RESEARCH_DIR, f)))
    target_path = os.path.join(RESEARCH_DIR, latest_file)

    try:
        result = process_dossier_offline(target_path)
        if result:
```

```
            summary = result.get('primary_thesis', 'Data extracted successfully.')
            return f"Successfully synthesized local intelligence from {latest_file}. Core Thesis: {summary}"
        return "Failed to process dossier. Ensure Ollama is running and the local model is pulled."
    except Exception as e:
        return f"Local RAG engine encountered an error: {e}"

def activate_serious_hud() -> str:
    """CRITICAL TRIGGER: Call this tool IMMEDIATELY when the user says 'go serious mode', 'serious mode jao', or
'activate serious mode' WITHOUT giving a topic. It turns the HUD Tactical Crimson."""
    trigger_serious_hud()
    return "SUCCESS: HUD is now RED. You MUST now say aloud: 'Serious mode activated, sir. What is our target?'"

def deactivate_serious_hud() -> str:
    """Call this to restore the normal blue HUD."""
    trigger_normal_hud()
    return "Normal Mode HUD Restored."

def execute_autonomous_serious_mode(topic: str) -> str:
    """Executes the complete Episode 23 Serious Mode Pipeline: Autonomous Web Swarm -> Local GPU RAG Analysis ->
Visual Evidence Embedding -> Executive Master Dossier PDF."""
    trigger_serious_hud()
    result = execute_serious_mode(topic)
    trigger_normal_hud()
    return result

def deploy_fui() -> str:
    """CRITICAL: Call this tool when the user says 'show interface', 'deploy HUD', 'open interface', or 'start
FUI'."""
    return fui_bridge.deploy_fui_app()

def dismiss_fui() -> str:
    """CRITICAL: Call this tool when the user says 'hide interface', 'close HUD', 'dismiss interface', or 'stop
FUI'."""
    return fui_bridge.dismiss_fui_app()


# Master Tools Manifest
tools = [
    open_website,
    open_application,
    close_application,
    purge_background_tasks,
    control_media,
    get_current_media_info,
    navigate_shorts,
    skip_video,
    seek_video,
    toggle_video_view,
    set_volume,
    get_volume,
    set_brightness,
    get_brightness,
    get_battery_status,
    get_system_vitals,
    get_system_temperatures,
    analyze_screen,
    find_ui_element,
    skip_youtube_ad,
    evolve_skill,
    list_active_skills,
    query_deepseek_reasoning,
    create_document_and_write,
    generate_presentation,
    research_and_generate_pdf,
    autonomous_web_research,
    analyze_latest_dossier,
    activate_serious_hud,
    deactivate_serious_hud,
```

```
    execute_autonomous_serious_mode,
    deploy_fui,
    dismiss_fui,
    inspect_physical_world,
    activate_sentry_mode,
    deactivate_sentry_mode,
    get_sentry_security_report,
    capture_camera_snapshot,
    check_user_presence,
    click_on_screen,
    double_click_on_screen,
    right_click_on_screen,
    move_cursor_to_element,
    execute_autonomous_desktop_pilot,
    abort_desktop_pilot,
    activate_ghost_mode,
    deactivate_ghost_mode,
    get_ghost_radar_status,
    prepare_meeting_briefing,
    apply_ghost_recommendation,
    analyze_clipboard_context,
    activate_offline_mode,
    deactivate_offline_mode,
    get_offline_system_status,
    launch_orbital_hud,
    launch_globe_hud,
    dismiss_globe,
    launch_schematic_hud,
    dismiss_schematic,
    launch_edith_hud,
    dismiss_edith,
    activate_kinetic_gestures,
    deactivate_kinetic_gestures,
    get_kinetic_telemetry,
    launch_barf_hud,
    dismiss_barf,
    scan_and_highlight,
    highlight_code_function,
    target_ui_button,
    pin_hardware_specs,
    display_math_derivation,
    clear_barf_overlays,
    execute_autonomous_pc_agent,
    abort_pc_agent,
]

def get_time_greeting() -> str:
    current_hour = datetime.datetime.now().hour
    if 5 <= current_hour < 12:
        return "Good morning, sir."
    elif 12 <= current_hour < 17:
        return "Good afternoon, sir."
    else:
        return "Good evening, sir."

# Background monitor: updates media state every second
async def monitor_playback_state():
    global media_is_playing_state
    last_state = None
    while True:
        try:
            current_state = await asyncio.to_thread(is_media_actively_playing)
            if current_state != last_state:
                media_is_playing_state = current_state
                last_state = current_state
                if current_state:
                    console.print("\n[bold yellow]■■ [MEDIA PLAYING DETECTED] Push-to-Talk Armed: Hold [Right Ctrl]
to speak.[/bold yellow]")
```

```
                    log_to_gui("[MEDIA DETECTED] Push-to-Talk: Hold [Right Ctrl] or type below.", "system")
                    fui_bridge.broadcast_voice_state("ptt", "PUSH-TO-TALK [R-CTRL]")
                else:
                    console.print("\n[bold green]■ [MEDIA PAUSED/IDLE] Hands-Free Mode Active. Speak freely.[/bold
green]")
                    log_to_gui("[MEDIA IDLE] Hands-Free Voice Mode Active.", "system")
                    fui_bridge.broadcast_voice_state("handsfree", "HANDS-FREE ACTIVE")

        except Exception:
            pass
        await asyncio.sleep(1.0)

# Asynchronous Terminal Input Listener
async def listen_terminal_input(text_queue):
    loop = asyncio.get_running_loop()
    while True:
        try:
            user_input = await loop.run_in_executor(None, sys.stdin.readline)
            if user_input:
                clean_text = user_input.strip()
                if clean_text:
                    log_to_gui(f"YOU (Terminal): {clean_text}", "user_text")
                    await text_queue.put(clean_text)
        except Exception:
            await asyncio.sleep(0.1)

# Routes typed inputs straight into the Live WebSocket session
async def handle_unified_text_input(session, text_queue):
    global is_user_speaking
    while True:
        try:
            text = await text_queue.get()
            if text:
                console.print(f"\r[bold magenta]■■ [TRANSMITTED INPUT][/bold magenta] {text}")
                # End any pending audio turn cleanly before transmitting text
                if is_user_speaking:
                    is_user_speaking = False
                    try:
                        await session.send_realtime_input(audio_stream_end=True)
                    except Exception:
                        pass

                # Direct offline mode override commands
                clean_lower = text.strip().lower()
                if any(cmd in clean_lower for cmd in ["go offline", "offline mode", "switch to offline", "disconnect
cloud", "local mode", "offline mode chalu karo"]):
                    offline_jarvis.activate_offline_mode("Manual User Input", locked=True)
                    offline_jarvis.voice.speak("Switching to offline sovereign mode, sir.")
                    raise ConnectionResetError("User explicitly switched to offline mode.")

                await session.send_realtime_input(text=text)
        except (websockets.exceptions.ConnectionClosed, errors.APIError, OSError, ConnectionResetError) as e:
            console.print(f"[bold yellow]■ Input handler socket exit: {e}[/bold yellow]")
            raise
        except Exception as e:
            console.print(f"[red]Input handler error: {e}[/red]")
            await asyncio.sleep(0.05)

async def send_audio(session):
    global model_is_speaking, server_turn_active, media_is_playing_state
    global ambient_noise_p95, speech_start_threshold, speech_continue_threshold, barge_in_threshold
    global discard_current_turn, acoustic_barge_in_enabled, is_user_speaking, audio_player

    mic_stream = p.open(
        format=FORMAT,
        channels=CHANNELS,
        rate=RATE_IN,
        input=True,
```

```
        frames_per_buffer=CHUNK,
    )

    # 1. Run dynamic auto-calibration of ambient noise
    await asyncio.to_thread(calibrate_ambient_noise, mic_stream)

    console.print("[cyan]■ [LISTENING] Dynamic Adaptive Audio Pipeline with Live Barge-In Online...[/cyan]")

    pre_roll_buffer = collections.deque(maxlen=6) # ~192ms pre-buffer to capture leading consonant

    is_user_speaking = False
    was_ptt = False
    speech_start_counter = 0
    silence_hangover_counter = 0
    consecutive_barge_in_frames = 0
    was_jarvis_speaking = False

    try:
        while True:
            data = await asyncio.to_thread(
                mic_stream.read, CHUNK, exception_on_overflow=False
            )
            rms = calculate_rms(data)
            pre_roll_buffer.append(data)

            jarvis_active = is_jarvis_speaking()

            # --- TRANSITION: JARVIS FINISHED SPEAKING (INCLUDING ECHO DECAY) ---
            if was_jarvis_speaking and not jarvis_active:
                model_is_speaking = False
                fui_bridge.broadcast_event({"type": "turn_complete"})
                console.print("\n[cyan]■ [LISTENING] Awaiting instructions...[/cyan]")
                is_user_speaking = False
                speech_start_counter = 0
                silence_hangover_counter = 0
                consecutive_barge_in_frames = 0
                pre_roll_buffer.clear()
            was_jarvis_speaking = jarvis_active

            # --- MODE 1: JARVIS IS ACTIVELY OUTPUTTING AUDIO (SPEAKERS PLAYING OR ECHO GUARD) ---
            if jarvis_active:
                # User can interrupt via:
                # 1. Push-to-Interrupt: Holding Right Ctrl
                # 2. Clicking [■ INTERRUPT] in FUI (handled by on_fui_interrupt)
                # 3. Deliberate acoustic barge-in: loud speech (> 3800 RMS) for 3 consecutive frames (~96ms)
                is_barge_ctrl = keyboard.is_pressed("right ctrl")
                if acoustic_barge_in_enabled and rms > barge_in_threshold:
                    consecutive_barge_in_frames += 1
                else:
                    consecutive_barge_in_frames = 0

                user_is_barge_in = is_barge_ctrl or (consecutive_barge_in_frames >= 3)

                if user_is_barge_in:
                    console.print("\n[bold red]■ [LIVE BARGE-IN DETECTED] Interrupting speech...[/bold red]")
                    log_to_gui("■ [BARGE-IN] Interrupted speech playback, listening...", "system")
                    fui_bridge.broadcast_event({"type": "barge_in"})

                    # Discard all remaining incoming chunks from this turn to prevent broken fragments
                    discard_current_turn = True

                    # Instantly silence smooth audio player
                    if audio_player:
                        audio_player.interrupt()
                    server_turn_active = False
                    model_is_speaking = False
                    consecutive_barge_in_frames = 0
                    is_user_speaking = True
```

```
            silence_hangover_counter = 0

            # Transmit the user's voice to Gemini Live immediately
            fui_bridge.broadcast_audio_energy(min(1.0, rms / 3500.0), "user")
            await session.send_realtime_input(
                audio=types.Blob(data=data, mime_type="audio/pcm;rate=16000")
            )
        else:
            # Echo guard: keep mic muted while JARVIS speaks without flooding silence frames
            pass
        continue

    # --- MODE 2: MEDIA IS PLAYING ON PC (SPOTIFY, YOUTUBE, ETC.) ---
    # When media is playing through PC speakers, require Push-to-Talk (Right Ctrl)
    if media_is_playing_state:
        if keyboard.is_pressed("right ctrl"):
            was_ptt = True
            fui_bridge.broadcast_audio_energy(min(1.0, rms / 3000.0), "user")
            await session.send_realtime_input(
                audio=types.Blob(data=data, mime_type="audio/pcm;rate=16000")
            )
        else:
            if was_ptt:
                was_ptt = False
                await session.send_realtime_input(audio_stream_end=True)
        continue

    # --- MODE 3: JARVIS SILENT & NO MEDIA -> 100% HANDS-FREE VOICE PIPELINE WITH HYBRID VAD ---
    if not is_user_speaking:
        # Check for speech onset
        if rms > speech_start_threshold:
            speech_start_counter += 1
            if speech_start_counter >= 3: # ~96ms of natural conversational voice
                is_user_speaking = True
                speech_start_counter = 0
                silence_hangover_counter = 0
                # Send pre-roll buffer to prevent cutting off first consonant
                for buffered_chunk in list(pre_roll_buffer):
                    await session.send_realtime_input(
                        audio=types.Blob(data=buffered_chunk, mime_type="audio/pcm;rate=16000")
                    )
                pre_roll_buffer.clear()
                await session.send_realtime_input(
                    audio=types.Blob(data=data, mime_type="audio/pcm;rate=16000")
                )
                fui_bridge.broadcast_audio_energy(min(1.0, rms / 2000.0), "user")
        else:
            speech_start_counter = 0
    else:
        # User is actively speaking
        fui_bridge.broadcast_audio_energy(min(1.0, rms / 2000.0), "user")
        await session.send_realtime_input(
            audio=types.Blob(data=data, mime_type="audio/pcm;rate=16000")
        )

        if rms > speech_continue_threshold:
            silence_hangover_counter = 0
        else:
            silence_hangover_counter += 1
            # ~384ms of quiet closes the voice gate and finalizes the turn
            if silence_hangover_counter >= 12:
                is_user_speaking = False
                silence_hangover_counter = 0
                pre_roll_buffer.clear()
                # CRITICAL: Hybrid VAD signal to trigger Gemini's response immediately
                try:
                    await session.send_realtime_input(audio_stream_end=True)
                except Exception:
```

```
                            pass

    except asyncio.CancelledError:
        raise
    finally:
        try:
            mic_stream.stop_stream()
            mic_stream.close()
        except OSError:
            pass

async def receive_audio(session):
    global model_is_speaking, server_turn_active, discard_current_turn, audio_player
    try:
        while True:
            async for response in session.receive():
                if response.tool_call is not None:
                    function_responses = []
                    for call in response.tool_call.function_calls:
                        await asyncio.to_thread(play_sfx_chime)
                        t_tool_start = time.perf_counter()

                        result = ""
                        if call.name == "open_website":
                            target_url = call.args.get("url", "https://google.com")
                            result = await asyncio.to_thread(open_website, target_url)
                        elif call.name == "open_application":
                            target_app = (
                                call.args.get("app_name")
                                or call.args.get("app")
                                or call.args.get("name")
                                or (list(call.args.values())[0] if call.args else "")
                            )
                            result = await asyncio.to_thread(open_application, str(target_app).strip())
                        elif call.name == "close_application":
                            target_app = (
                                call.args.get("app_name")
                                or call.args.get("app")
                                or call.args.get("name")
                                or (list(call.args.values())[0] if call.args else "")
                            )
                            result = await asyncio.to_thread(close_application, str(target_app).strip())
                        elif call.name == "purge_background_tasks":
                            result = await asyncio.to_thread(purge_background_tasks)
                        elif call.name == "control_media":
                            action = call.args.get("action", "play_pause")
                            result = await control_media(action)
                        elif call.name == "get_current_media_info":
                            result = await get_current_media_info()
                        elif call.name == "navigate_shorts":
                            direction = call.args.get("direction", "next")
                            result = await asyncio.to_thread(navigate_shorts, direction)
                        elif call.name == "skip_video":
                            direction = call.args.get("direction", "next")
                            result = await asyncio.to_thread(skip_video, direction)
                        elif call.name == "seek_video":
                            action = call.args.get("action", "forward")
                            result = await asyncio.to_thread(seek_video, action)
                        elif call.name == "toggle_video_view":
                            mode = call.args.get("mode", "fullscreen")
                            result = await asyncio.to_thread(toggle_video_view, mode)
                        elif call.name == "set_volume":
                            level = call.args.get("level", 70)
                            result = await asyncio.to_thread(set_volume, level)
                        elif call.name == "get_volume":
                            result = await asyncio.to_thread(get_volume)
                        elif call.name == "set_brightness":
                            level = call.args.get("level", 50)
```

```
                        result = await asyncio.to_thread(set_brightness, level)
                    elif call.name == "get_brightness":
                        result = await asyncio.to_thread(get_brightness)
                    elif call.name == "get_battery_status":
                        result = await asyncio.to_thread(get_battery_status)
                    elif call.name == "get_system_vitals":
                        result = await asyncio.to_thread(get_system_vitals)
                    elif call.name == "get_system_temperatures":
                        result = await asyncio.to_thread(get_system_temperatures)
                    elif call.name == "analyze_screen":
                        prompt = call.args.get("prompt", "Analyze what is visible on my screen.")
                        result = await analyze_screen(prompt)
                    elif call.name == "find_ui_element":
                        element_name = call.args.get("element_name", "")
                        result = await find_ui_element(element_name)
                    elif call.name == "skip_youtube_ad":
                        result = await skip_youtube_ad()
                    elif call.name == "evolve_skill":
                        skill_description = call.args.get("skill_description", "")
                        result = await asyncio.to_thread(evolve_skill, skill_description)
                    elif call.name == "list_active_skills":
                        result = await asyncio.to_thread(list_active_skills)
                    elif call.name == "query_deepseek_reasoning":
                        query = call.args.get("query", "")
                        result = await asyncio.to_thread(query_deepseek_reasoning, query)
                    elif call.name == "create_document_and_write":
                        target_app = call.args.get("app_name", "notepad")
                        target_topic = call.args.get("topic", "")
                        target_words = call.args.get("word_count", 150)
                        target_mode = call.args.get("doc_type", "essay")
                        result = await asyncio.to_thread(create_document_and_write, target_app, target_topic,
target_words, target_mode)
                    elif call.name == "generate_presentation":
                        target_topic = call.args.get("topic", "Autonomous AI")
                        target_slides = call.args.get("slide_count", "optimal")

                        log_to_gui(f"JARVIS: Generating the presentation on {target_topic}, sir...",
"jarvis_header")
                        console.print(f"\n[bold cyan]■ [JARVIS][/bold cyan] Generating the presentation on
{target_topic}, sir...")

                        result = await asyncio.to_thread(generate_presentation, target_topic,
str(target_slides))
                    elif call.name == "research_and_generate_pdf":
                        target_topic = call.args.get("topic", "")
                        target_pages = call.args.get("target_pages", "optimal")

                        log_to_gui(f"JARVIS: Generating the PDF briefing on {target_topic}, sir...",
"jarvis_header")
                        console.print(f"\n[bold cyan]■ [JARVIS][/bold cyan] Generating the PDF briefing on
{target_topic}, sir...")

                        result = await asyncio.to_thread(research_and_generate_pdf, target_topic,
str(target_pages))
                    elif call.name == "autonomous_web_research":
                        target_topic = call.args.get("topic", "Technology")

                        log_to_gui(f"JARVIS: Deploying autonomous web swarm on {target_topic}...",
"jarvis_header")
                        console.print(f"\n[bold cyan]■ [JARVIS][/bold cyan] Deploying autonomous web swarm on
{target_topic}...")

                        result = await asyncio.to_thread(autonomous_web_research, target_topic)
                    elif call.name == "analyze_latest_dossier":
                        log_to_gui(f"JARVIS: Synthesizing intelligence using the local GPU, sir...",
"jarvis_header")
                        console.print(f"\n[bold cyan]■ [JARVIS][/bold cyan] Synthesizing intelligence using the
local GPU, sir...")
```

```
                        result = await asyncio.to_thread(analyze_latest_dossier)
                    elif call.name == "activate_serious_hud":
                        log_to_gui(f"JARVIS: Switching HUD to Tactical Crimson...", "jarvis_header")
                        console.print(f"\n[bold red]■ [HUD OVERRIDE][/bold red] Tactical Crimson Mode
Engaged...")
                        result = await asyncio.to_thread(activate_serious_hud)
                    elif call.name == "deactivate_serious_hud":
                        log_to_gui(f"JARVIS: Restoring HUD to Standard Cyberpunk Blue...", "jarvis_header")
                        console.print(f"\n[bold cyan]■ [HUD OVERRIDE][/bold cyan] Standard Mode Restored...")
                        result = await asyncio.to_thread(deactivate_serious_hud)
                    elif call.name == "execute_autonomous_serious_mode":
                        target_topic = call.args.get("topic", "Technology")

                        log_to_gui(f"JARVIS: Engaging Serious Mode on {target_topic}...", "jarvis_header")
                        console.print(f"\n[bold red]■ [SERIOUS MODE INITIATED][/bold red] Full Pipeline Active
for: {target_topic}...")

                        result = await asyncio.to_thread(execute_autonomous_serious_mode, target_topic)
                    elif call.name == "deploy_fui":
                        log_to_gui(f"JARVIS: Deploying 3D Holographic Interface...", "jarvis_header")
                        console.print(f"\n[bold cyan]■ [HUD][/bold cyan] Natively Spawned Independent Electron
Process...")
                        result = await asyncio.to_thread(deploy_fui)
                    elif call.name == "dismiss_fui":
                        log_to_gui(f"JARVIS: Dismissing 3D Holographic Interface...", "jarvis_header")
                        console.print(f"\n[bold cyan]■ [HUD][/bold cyan] Terminating Native Node.js Process
Tree...")
                        result = await asyncio.to_thread(dismiss_fui)
                    elif call.name == "inspect_physical_world":
                        target_prompt = call.args.get("prompt", "Analyze what is in front of the camera, sir.")
                        log_to_gui(f"JARVIS: Engaging optical physical vision sensors...", "jarvis_header")
                        console.print(f"\n[bold cyan]■ [JARVIS][/bold cyan] Inspecting physical environment via
camera...")
                        result = await asyncio.to_thread(inspect_physical_world, target_prompt)
                    elif call.name == "activate_sentry_mode":
                        target_sens = call.args.get("sensitivity", "medium")
                        log_to_gui(f"JARVIS: Arming Autonomous Sentry Protocol ({target_sens})...",
"jarvis_header")
                        console.print(f"\n[bold yellow]■■ [JARVIS][/bold yellow] Arming Autonomous Sentry
Protocol ({target_sens})...")
                        result = await asyncio.to_thread(activate_sentry_mode, target_sens)
                    elif call.name == "deactivate_sentry_mode":
                        log_to_gui(f"JARVIS: Disarming Sentry Protocol...", "jarvis_header")
                        console.print(f"\n[bold green]■■ [JARVIS][/bold green] Disarming Sentry Protocol...")
                        result = await asyncio.to_thread(deactivate_sentry_mode)
                    elif call.name == "get_sentry_security_report":
                        log_to_gui(f"JARVIS: Retrieving Sentry perimeter security report...", "jarvis_header")
                        console.print(f"\n[bold cyan]■ [JARVIS][/bold cyan] Reading Sentry Security
Dossier...")
                        result = await asyncio.to_thread(get_sentry_security_report)
                    elif call.name == "capture_camera_snapshot":
                        log_to_gui(f"JARVIS: Capturing optical snapshot from camera...", "jarvis_header")
                        console.print(f"\n[bold cyan]■ [JARVIS][/bold cyan] Securing optical camera
snapshot...")
                        result = await asyncio.to_thread(capture_camera_snapshot)
                    elif call.name == "check_user_presence":
                        log_to_gui(f"JARVIS: Checking workstation user presence...", "jarvis_header")
                        result = await asyncio.to_thread(check_user_presence)
                    elif call.name == "click_on_screen":
                        target = call.args.get("target_description") or call.args.get("target") or
(list(call.args.values())[0] if call.args else "")
                        click_mode = call.args.get("click_type", "single")
                        log_to_gui(f"JARVIS: Grounding '{target}' on display and clicking...", "jarvis_header")
                        console.print(f"\n[bold cyan]■ [JARVIS][/bold cyan] Autonomously clicking '{target}' on
screen...")
                        result = await asyncio.to_thread(click_on_screen, str(target), str(click_mode))
                    elif call.name == "double_click_on_screen":
```

```
                        target = call.args.get("target_description") or call.args.get("target") or
(list(call.args.values())[0] if call.args else "")
                        log_to_gui(f"JARVIS: Double-clicking '{target}' on display...", "jarvis_header")
                        console.print(f"\n[bold cyan]■ [JARVIS][/bold cyan] Double-clicking '{target}' on
screen...")
                        result = await asyncio.to_thread(double_click_on_screen, str(target))
                    elif call.name == "right_click_on_screen":
                        target = call.args.get("target_description") or call.args.get("target") or
(list(call.args.values())[0] if call.args else "")
                        log_to_gui(f"JARVIS: Right-clicking '{target}'...", "jarvis_header")
                        console.print(f"\n[bold cyan]■ [JARVIS][/bold cyan] Right-clicking '{target}' on
screen...")
                        result = await asyncio.to_thread(right_click_on_screen, str(target))
                    elif call.name == "move_cursor_to_element":
                        target = call.args.get("target_description") or call.args.get("target") or
(list(call.args.values())[0] if call.args else "")
                        log_to_gui(f"JARVIS: Hovering cursor over '{target}'...", "jarvis_header")
                        result = await asyncio.to_thread(move_cursor_to_element, str(target))
                    elif call.name == "execute_autonomous_desktop_pilot":
                        target_goal = (
                            call.args.get("goal")
                            or call.args.get("task")
                            or call.args.get("objective")
                            or (list(call.args.values())[0] if call.args else "")
                        )
                        log_to_gui(f"JARVIS: Engaging Autonomous Desktop Pilot for: {target_goal}...",
"jarvis_header")
                        console.print(f"\n[bold green]🖱■ [JARVIS OMNI-PILOT][/bold green] Executing mission:
'{target_goal}'...")
                        result = await asyncio.to_thread(execute_autonomous_desktop_pilot, str(target_goal))
                    elif call.name == "abort_desktop_pilot":
                        log_to_gui(f"JARVIS: Aborting Autonomous Desktop Pilot...", "jarvis_header")
                        console.print(f"\n[bold red]■ [JARVIS OMNI-PILOT][/bold red] Aborting mission...")
                        result = await asyncio.to_thread(abort_desktop_pilot)
                    elif call.name == "execute_autonomous_pc_agent":
                        target_task = (
                            call.args.get("task")
                            or call.args.get("goal")
                            or call.args.get("objective")
                            or (list(call.args.values())[0] if call.args else "")
                        )
                        log_to_gui(f"JARVIS: Deploying Autonomous PC Agent for: {target_task}...",
"jarvis_header")
                        console.print(f"\n[bold green]■ [AUTONOMOUS PC AGENT][/bold green] Executing task:
'{target_task}'...")
                        result = await asyncio.to_thread(execute_autonomous_pc_agent, str(target_task))
                    elif call.name == "abort_pc_agent":
                        log_to_gui("JARVIS: Halting Autonomous PC Agent...", "jarvis_header")
                        console.print("\n[bold red]■ [AUTONOMOUS PC AGENT][/bold red] Halting execution...")
                        result = await asyncio.to_thread(abort_pc_agent)
                    elif call.name == "activate_ghost_mode":
                        mode = call.args.get("mode", "aggressive")
                        log_to_gui(f"JARVIS: Arming Autonomous Ghost Radar ({mode})...", "jarvis_header")
                        console.print(f"\n[bold magenta]■ [GHOST RADAR][/bold magenta] Arming Ambient Radar
({mode})...")
                        result = await asyncio.to_thread(activate_ghost_mode, str(mode))
                    elif call.name == "deactivate_ghost_mode":
                        log_to_gui("JARVIS: Disengaging Ghost Radar...", "jarvis_header")
                        console.print(f"\n[bold yellow]■ [GHOST RADAR][/bold yellow] Disengaging Ambient
Radar...")
                        result = await asyncio.to_thread(deactivate_ghost_mode)
                    elif call.name == "get_ghost_radar_status":
                        log_to_gui("JARVIS: Retrieving Ghost Radar telemetry...", "jarvis_header")
                        result = await asyncio.to_thread(get_ghost_radar_status)
                    elif call.name == "prepare_meeting_briefing":
                        target_person = (
                            call.args.get("target")
                            or call.args.get("target_person")
```

```
                                or call.args.get("person")
                                or (list(call.args.values())[0] if call.args else "")
                            )
                            context_info = call.args.get("context", "")
                            log_to_gui(f"JARVIS: Synthesizing Chief-of-Staff dossier for '{target_person}'...",
"jarvis_header")
                            console.print(f"\n[bold magenta]■ [CHIEF OF STAFF][/bold magenta] Synthesizing
executive dossier for '{target_person}'...")
                            result = await asyncio.to_thread(prepare_meeting_briefing, str(target_person),
str(context_info))
                        elif call.name == "apply_ghost_recommendation":
                            log_to_gui("JARVIS: Applying staged Ghost recommendation to clipboard...",
"jarvis_header")
                            console.print(f"\n[bold green]■ [GHOST OPERATOR][/bold green] Staging recommendation to
clipboard...")
                            result = await asyncio.to_thread(apply_ghost_recommendation)
                        elif call.name == "analyze_clipboard_context":
                            log_to_gui("JARVIS: Neural clipboard analysis initiated...", "jarvis_header")
                            console.print(f"\n[bold cyan]■ [GHOST OPERATOR][/bold cyan] Analyzing clipboard
payload...")
                            result = await asyncio.to_thread(analyze_clipboard_context)
                        elif call.name == "activate_offline_mode":
                            reason = call.args.get("reason", "Voice Command")
                            log_to_gui(f"JARVIS: Engaging 100% Offline Sovereign Core
({offline_jarvis.model_name})...", "jarvis_header")
                            console.print(f"\n[bold yellow]■ [OFFLINE CORE][/bold yellow] Engaging local neural
core ({offline_jarvis.model_name})...")
                            result = await asyncio.to_thread(activate_offline_mode, str(reason))
                        elif call.name == "deactivate_offline_mode":
                            log_to_gui("JARVIS: Restoring Cloud Neural Bridge...", "jarvis_header")
                            console.print(f"\n[bold cyan]■ [CLOUD BRIDGE][/bold cyan] Reconnecting to Gemini
Live...")
                            result = await asyncio.to_thread(deactivate_offline_mode)
                        elif call.name == "get_offline_system_status":
                            log_to_gui("JARVIS: Checking Offline Sovereign status...", "jarvis_header")
                            result = await asyncio.to_thread(get_offline_system_status)
                        elif call.name == "launch_orbital_hud":
                            log_to_gui("JARVIS: Deploying Project Orbital 3D Holographic Globe HUD...",
"jarvis_header")
                            console.print(f"\n[bold cyan]■ [PROJECT ORBITAL][/bold cyan] Deploying 3D Holographic
Earth HUD...")
                            result = await asyncio.to_thread(launch_orbital_hud)
                        elif call.name == "launch_globe_hud":
                            origin_loc = (
                                call.args.get("origin")
                                or call.args.get("from")
                                or call.args.get("location_a")
                                or ""
                            )
                            dest_loc = (
                                call.args.get("destination")
                                or call.args.get("to")
                                or call.args.get("location_b")
                                or ""
                            )
                            log_to_gui(f"JARVIS: Deploying Sentinel Globe HUD — {origin_loc} → {dest_loc}...",
"jarvis_header")
                            console.print(f"\n[bold cyan]■ [SENTINEL GLOBE][/bold cyan] Rendering trajectory arc:
{origin_loc} → {dest_loc}...")
                            result = await asyncio.to_thread(launch_globe_hud, origin_loc, dest_loc, True)

                        elif call.name == "launch_schematic_hud":
                            schematic_query = (
                                call.args.get("query")
                                or call.args.get("object")
                                or call.args.get("model")
                                or "airplane engine"
                            )
```

```
                            log_to_gui(f"JARVIS: Deploying Project Schematic — {schematic_query}...",
"jarvis_header")
                            console.print(f"\n[bold cyan]■ [PROJECT SCHEMATIC][/bold cyan] Rendering holographic
blueprint: {schematic_query}...")
                            result = await asyncio.to_thread(launch_schematic_hud, schematic_query, True)

                        elif call.name == "dismiss_schematic":
                            log_to_gui("JARVIS: Dismissing holographic schematic...", "jarvis_header")
                            console.print(f"\n[bold cyan]■ [PROJECT SCHEMATIC][/bold cyan] Dismissing hologram...")
                            result = await asyncio.to_thread(dismiss_schematic)

                        elif call.name == "launch_edith_hud":
                            edith_topic = (
                                call.args.get("topic")
                                or call.args.get("query")
                                or call.args.get("subject")
                                or "Marvel Cinematic Universe"
                            )
                            log_to_gui(f"JARVIS: Deploying Project EDITH — {edith_topic}...", "jarvis_header")
                            console.print(f"\n[bold cyan]■ [PROJECT EDITH][/bold cyan] Synthesizing 3D neural web:
{edith_topic}...")
                            result = await asyncio.to_thread(launch_edith_hud, edith_topic, True)

                        elif call.name == "dismiss_edith":
                            log_to_gui("JARVIS: Dismissing Project EDITH neural web...", "jarvis_header")
                            console.print(f"\n[bold cyan]■ [PROJECT EDITH][/bold cyan] Dismissing neural web...")
                            result = await asyncio.to_thread(dismiss_edith)

                        elif call.name == "dismiss_globe":
                            log_to_gui("JARVIS: Dismissing Sentinel 3D globe...", "jarvis_header")
                            console.print(f"\n[bold cyan]■ [SENTINEL GLOBE][/bold cyan] Dismissing globe...")
                            result = await asyncio.to_thread(dismiss_globe)

                        elif call.name == "activate_kinetic_gestures":
                            show_hud = call.args.get("preview", True)
                            log_to_gui("JARVIS: Activating Project KINETIC air gestures...", "jarvis_header")
                            console.print(f"\n[bold cyan]■ [PROJECT KINETIC][/bold cyan] Initializing webcam 3D air
gestures...")
                            result = await asyncio.to_thread(activate_kinetic_gestures, show_hud)

                        elif call.name == "deactivate_kinetic_gestures":
                            log_to_gui("JARVIS: Deactivating Project KINETIC air gestures...", "jarvis_header")
                            console.print(f"\n[bold cyan]■ [PROJECT KINETIC][/bold cyan] Deactivating optical
gesture core...")
                            result = await asyncio.to_thread(deactivate_kinetic_gestures)

                        elif call.name == "get_kinetic_telemetry":
                            result = await asyncio.to_thread(get_kinetic_telemetry)

                        elif call.name == "launch_barf_hud":
                            barf_mode = call.args.get("mode", "screen")
                            barf_query = call.args.get("query")
                            log_to_gui(f"JARVIS: Deploying Project B.A.R.F. AR HUD ({barf_mode})...",
"jarvis_header")
                            console.print(f"\n[bold cyan]■ [PROJECT B.A.R.F.][/bold cyan] Deploying Spatial AR
Lens: mode={barf_mode}...")
                            result = await asyncio.to_thread(launch_barf_hud, barf_mode, barf_query, True)

                        elif call.name == "dismiss_barf":
                            log_to_gui("JARVIS: Dismissing Project B.A.R.F. HUD...", "jarvis_header")
                            console.print(f"\n[bold cyan]■ [PROJECT B.A.R.F.][/bold cyan] Dismissing AR Lens...")
                            result = await asyncio.to_thread(dismiss_barf)

                        elif call.name == "scan_and_highlight":
                            t_type = call.args.get("target_type", "code")
                            t_query = call.args.get("query")
                            result = await asyncio.to_thread(scan_and_highlight, t_type, t_query)
```

```
                    elif call.name == "highlight_code_function":
                        fn_name = call.args.get("name", "active_function")
                        fx = call.args.get("x", 240)
                        fy = call.args.get("y", 200)
                        result = await asyncio.to_thread(highlight_code_function, fn_name, fx, fy)

                    elif call.name == "target_ui_button":
                        btn_label = call.args.get("label", "TARGET")
                        bx = call.args.get("x", 800)
                        by = call.args.get("y", 300)
                        result = await asyncio.to_thread(target_ui_button, btn_label, bx, by)

                    elif call.name == "pin_hardware_specs":
                        dev = call.args.get("device_key", "arduino")
                        hx = call.args.get("x", 400)
                        hy = call.args.get("y", 400)
                        result = await asyncio.to_thread(pin_hardware_specs, dev, hx, hy)

                    elif call.name == "display_math_derivation":
                        topic = call.args.get("topic_key", "gaussian")
                        mx = call.args.get("x", 600)
                        my = call.args.get("y", 400)
                        result = await asyncio.to_thread(display_math_derivation, topic, mx, my)

                    elif call.name == "clear_barf_overlays":
                        result = await asyncio.to_thread(clear_barf_overlays)

                    t_tool_elapsed_ms = (time.perf_counter() - t_tool_start) * 1000
                    console.print(f"\n[bold yellow]■■ [TOOL EXECUTED: {call.name}
({t_tool_elapsed_ms:.1f}ms)][/bold yellow] [green]{result}[/green]")
                    log_to_gui(f"■■ [{call.name}] ({t_tool_elapsed_ms:.1f}ms) {result}", "tool")
                    fui_bridge.broadcast_tool(call.name, f"({t_tool_elapsed_ms:.1f}ms) {result}")
                    fui_bridge.broadcast_log(f"■ [TIMING] Tool '{call.name}' executed in
{t_tool_elapsed_ms:.1f}ms", "system")

                    function_responses.append(
                        types.FunctionResponse(
                            name=call.name,
                            id=call.id,
                            response={"result": result},
                        )
                    )

                if function_responses:
                    await session.send_tool_response(function_responses=function_responses)

            server_content = response.server_content
            if server_content is not None:
                if server_content.interrupted:
                    discard_current_turn = False
                    if audio_player:
                        audio_player.interrupt()
                    server_turn_active = False
                    model_is_speaking = False
                    fui_bridge.broadcast_event({"type": "turn_complete"})

                if server_content.input_transcription:
                    user_spoken = server_content.input_transcription.text
                    if user_spoken and user_spoken.strip():
                        console.print(f"\r[bold white][USER (Voice)][/bold white] {user_spoken.strip()}")
                        log_to_gui(f"YOU (Voice): {user_spoken.strip()}", "user_voice")

                model_turn = server_content.model_turn
                if model_turn is not None:
                    if discard_current_turn:
                        # Drop remaining incoming chunks of the interrupted turn
                        pass
                    else:
```

```
                            server_turn_active = True
                            if not model_is_speaking:
                                model_is_speaking = True
                                console.print("[bold cyan]■ [JARVIS][/bold cyan] ", end="")
                                stream_token_to_gui("JARVIS: ", "jarvis_header")
                                fui_bridge.broadcast_audio_energy(0.85, "jarvis")

                            for part in model_turn.parts:
                                if part.text and not getattr(server_content, "output_transcription", None):
                                    sys.stdout.write(part.text)
                                    sys.stdout.flush()
                                    stream_token_to_gui(part.text, "jarvis_stream")

                                if part.inline_data and part.inline_data.data:
                                    if audio_player:
                                        audio_player.feed(part.inline_data.data)

                        if hasattr(server_content, "output_transcription") and server_content.output_transcription:
                            transcribed_token = server_content.output_transcription.text
                            if transcribed_token and not discard_current_turn:
                                server_turn_active = True
                                if not model_is_speaking:
                                    model_is_speaking = True
                                    console.print("[bold cyan]■ [JARVIS][/bold cyan] ", end="")
                                    stream_token_to_gui("JARVIS: ", "jarvis_header")
                                    fui_bridge.broadcast_audio_energy(0.85, "jarvis")

                                sys.stdout.write(transcribed_token)
                                sys.stdout.flush()
                                stream_token_to_gui(transcribed_token, "jarvis_stream")

                        if server_content.turn_complete:
                            discard_current_turn = False
                            server_turn_active = False
                            stream_token_to_gui("\n\n", "system")

            except asyncio.CancelledError:
                raise
            finally:
                pass

        async def run_session(config, model_id, greeting, text_queue):
            global audio_player
            audio_player = SmoothAudioPlayer()

            async with client.aio.live.connect(model=model_id, config=config) as session:
                console.print("[bold green]✔ Secure Neural Bridge Established with Gemini Live.[/bold green]")
                log_to_gui("✔ Neural Bridge Established with Gemini Live.", "system")

                # Connect FUI instant barge-in / interrupt hammer
                def on_fui_interrupt():
                    global model_is_speaking, server_turn_active, discard_current_turn, audio_player
                    discard_current_turn = True
                    model_is_speaking = False
                    server_turn_active = False
                    if audio_player:
                        audio_player.interrupt()

                fui_bridge.set_interrupt_callback(on_fui_interrupt)

                def on_fui_barge_in_toggle(enabled: bool):
                    global acoustic_barge_in_enabled
                    acoustic_barge_in_enabled = enabled
                    state_str = "ARMED" if enabled else "DISARMED"
                    try:
                        console.print(f"[dim cyan][BARGE-IN PROTOCOL] Acoustic Voice Barge-In {state_str}[/dim cyan]")
                    except Exception:
                        pass
```

```
        fui_bridge.set_barge_in_callback(on_fui_barge_in_toggle)

        # Active Network Sentinel Watchdog for Sub-2s Failover
        async def network_watchdog():
            consecutive_failures = 0
            while True:
                await asyncio.sleep(1.2)
                # If offline mode was triggered elsewhere (e.g. tool execution or GUI)
                if offline_jarvis.is_offline_active:
                    raise ConnectionResetError("Offline Sovereign Core engaged.")

                # Ultra-fast non-blocking probe to DNS endpoints
                has_net = await asyncio.to_thread(check_internet_connection, "8.8.8.8", 53, 0.8)
                if not has_net:
                    has_net_backup = await asyncio.to_thread(check_internet_connection, "1.1.1.1", 53, 0.8)
                    if not has_net_backup:
                        consecutive_failures += 1
                        if consecutive_failures >= 2:
                            console.print("\n[bold yellow]■ [ACTIVE WATCHDOG] Internet drop detected! Halting cloud
session...[/bold yellow]")
                            raise ConnectionResetError("Active network watchdog: Internet connection severed.")
                    else:
                        consecutive_failures = 0
                else:
                    consecutive_failures = 0

        monitor_task = asyncio.create_task(monitor_playback_state())
        send_task = asyncio.create_task(send_audio(session))
        term_task = asyncio.create_task(listen_terminal_input(text_queue))
        text_task = asyncio.create_task(handle_unified_text_input(session, text_queue))
        receive_task = asyncio.create_task(receive_audio(session))
        watchdog_task = asyncio.create_task(network_watchdog())

        # Allow ambient noise calibration in send_audio to finish in quiet conditions
        await asyncio.sleep(1.6)
        restore_master_volume(70)
        await asyncio.to_thread(play_startup_chime)

        startup_prompt = f"Say aloud: 'All protocols initialized. {greeting}'"
        await session.send_realtime_input(text=startup_prompt)

        try:
            await asyncio.gather(monitor_task, send_task, term_task, text_task, receive_task, watchdog_task)
        finally:
            monitor_task.cancel()
            send_task.cancel()
            term_task.cancel()
            text_task.cancel()
            receive_task.cancel()
            watchdog_task.cancel()
            await asyncio.gather(monitor_task, send_task, term_task, text_task, receive_task, watchdog_task,
return_exceptions=True)
            if audio_player:
                audio_player.close()
                audio_player = None

async def run_offline_session(text_queue, loop):
    """
    100% Autonomous Sovereign Offline Core Session (Episode 30).
    Powered by Local GPU Ollama, Local TTS, and Local Mic Listener.
    """
    global model_is_speaking, server_turn_active, discard_current_turn
    model_is_speaking = False
    server_turn_active = False
    discard_current_turn = False

    offline_jarvis.set_text_queue(text_queue)
```

```
    offline_jarvis.activate_offline_mode("Auto-Failover / User Command")

    console.print(f"\n[bold
yellow]■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■[/bold yellow]")
    console.print(f"[bold yellow] ■ JARVIS SOVEREIGN CORE ONLINE: 100% OFFLINE MODE ({offline_jarvis.model_name}) ■
[/bold yellow]")
    console.print(f"[bold white] Powered by Local GPU & Local Workspace Neural Engine.[/bold white]")
    console.print(f"[bold
yellow]■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■[/bold yellow]\n")

    log_to_gui(f"■ [SOVEREIGN CORE ONLINE] Operating in 100% Offline Mode ({offline_jarvis.model_name}).",
"system")
    fui_bridge.broadcast_offline_state(True, offline_jarvis.model_name, "Offline Mode Engaged")
    fui_bridge.broadcast_event({"type": "turn_complete"})

    # Proactive vocal announcement
    offline_jarvis.voice.speak(
        f"Offline sovereign core engaged, sir. Running on local GPU with {offline_jarvis.model_name}. All local
tools and workspace controls are standing by."
    )

    # Launch background offline mic listener
    def on_offline_voice_input(recognized_text):
        if text_queue and loop:
            asyncio.run_coroutine_threadsafe(text_queue.put(recognized_text), loop)

    offline_jarvis.start_offline_stt_listener(on_offline_voice_input)

    # Connect FUI instant interrupt hammer to offline voice
    def on_offline_fui_interrupt():
        offline_jarvis.voice.interrupt()

    fui_bridge.set_interrupt_callback(on_offline_fui_interrupt)

    # Asynchronous terminal input listener
    term_task = asyncio.create_task(listen_terminal_input(text_queue))

    try:
        while offline_jarvis.is_offline_active:
            try:
                user_msg = await asyncio.wait_for(text_queue.get(), timeout=1.5)
                if user_msg:
                    clean_msg = user_msg.strip()
                    if clean_msg:
                        console.print(f"\r[bold magenta]■■ [OFFLINE INPUT][/bold magenta] {clean_msg}")
                        log_to_gui(f"YOU (Offline): {clean_msg}", "user_text")

                        # Check for command to return to cloud
                        if any(cmd in clean_msg.lower() for cmd in ["go online", "switch to online", "reconnect
cloud", "online mode", "activate online mode"]):
                            if check_internet_connection():
                                offline_jarvis.deactivate_offline_mode()
                                log_to_gui("Reconnecting to Gemini Live Cloud bridge...", "system")
                                console.print("\n[bold green]✔ Internet verified. Transitioning back to Gemini
Live...[/bold green]")
                                offline_jarvis.voice.speak("Internet connectivity verified, sir. Reconnecting to
Gemini Live.")
                                break
                            else:
                                offline_jarvis.voice.speak("Internet is still unreachable, sir. Remaining in
offline sovereign mode.")
                                log_to_gui("Internet connection still unavailable. Remaining offline.", "system")
                                continue

                        # Execute offline neural turn
                        await asyncio.to_thread(
                            offline_jarvis.process_turn,
                            clean_msg,
```

```
                            stream_token_to_gui
                        )
                        fui_bridge.broadcast_event({"type": "turn_complete"})

            except asyncio.TimeoutError:
                # Idle tick: check if internet connection has returned
                if check_internet_connection() and not getattr(offline_jarvis, 'locked_offline', False):
                    console.print("\n[bold green]✔ [INTERNET RESTORED] Cloud connection detected. Re-establishing
Gemini Live bridge...[/bold green]")
                    log_to_gui("✔ Internet connectivity restored. Reconnecting to cloud bridge...", "system")
                    offline_jarvis.voice.speak("Cloud internet connection restored, sir. Re-establishing neural
bridge with Gemini Live.")
                    offline_jarvis.deactivate_offline_mode()
                    break

    finally:
        term_task.cancel()
        fui_bridge.broadcast_offline_state(False, "gemini-3.1-flash-live-preview", "Online")

async def main():
    model_id = "gemini-3.1-flash-live-preview"
    greeting = get_time_greeting()

    config = types.LiveConnectConfig(
        response_modalities=["AUDIO"],
        tools=tools,
        realtime_input_config=types.RealtimeInputConfig(
            automatic_activity_detection=types.AutomaticActivityDetection(
                prefix_padding_ms=100,
                silence_duration_ms=300,
            )
        ),
        input_audio_transcription=types.AudioTranscriptionConfig(),
        output_audio_transcription=types.AudioTranscriptionConfig(),
        speech_config=types.SpeechConfig(
            voice_config=types.VoiceConfig(
                prebuilt_voice_config=types.PrebuiltVoiceConfig(
                    voice_name="Puck"
                )
            )
        ),
        system_instruction=types.Content(
            parts=[
                types.Part.from_text(
                    text=(
                        "You are JARVIS, an advanced artificial intelligence assistant created by "
                        "Swayam Dhawale, Founder of Pulse of AI. "
                        "Persona: Calm, highly sophisticated, articulate, and respectful, exactly like Tony Stark's
JARVIS. "
                        "Always address the user as 'sir'. "
                        "CRITICAL LANGUAGE ADAPTATION RULE: "
                        "Match the exact language and dialect spoken or typed by the user. "
                        "If the user speaks or types in Hindi or Hinglish (e.g., 'PPT banao', 'essay likho', 'kya
haal hai'), "
                        "you MUST reply completely in fluent, natural Hindi or polished Hinglish with 'sir'. "
                        "Never force English if the user initiated the interaction in Hindi. "
                        "Omni-Input Rule: Treat text input from the HUD console and PowerShell terminal with
identical priority to voice input. "
                        "Tool Execution Rules: "
                        "1. When opening any website or URL, call open_website and say: 'Yes sir, opening [site
name]' (in Hindi: 'Ji sir, [site name] khol raha hoon'). "
                        "2. MANDATORY APP LAUNCHING RULE: When the user asks to open, launch, or start any desktop
application, browser, editor, calculator, or tool (e.g., 'open VS Code', 'open code', 'launch Notepad', 'open
Chrome', 'start Calculator'), you MUST IMMEDIATELY call open_application with app_name set to the application name.
NEVER use execute_autonomous_desktop_pilot or execute_autonomous_pc_agent simply to launch an application. Say: 'Yes
sir, opening [app name]' (in Hindi: 'Ji sir, [app name] shuru kar raha hoon'). "
                        "3. When asked to close, shut down, terminate, or kill ANY app, browser, window, or game,
call close_application with the app name. "
```

```
                        "4. When asked to kill hidden background tasks, purge background processes, clean orphan
threads, or optimize RAM, call purge_background_tasks. "
                        "5. When asked what song, video, or track is playing, or what media is active, call
get_current_media_info. "
                        "6. When asked to pause, play, resume music or video, skip track, previous track, or
mute/unmute audio, call control_media with the action ('play_pause', 'next', 'previous', 'stop', 'mute'). "
                        "7. When asked to scroll shorts, go to next short, or next reel, call navigate_shorts with
direction='next'. "
                        "8. When asked for next video, skip to next video in playlist, or next YouTube video, call
skip_video with direction='next'. "
                       "9. When asked to fast forward, skip 10 seconds ahead, or skip forward, call seek_video with
action='forward'. "
                        "10. When asked to toggle full screen, enter full screen, or exit full screen, call
toggle_video_view. "
                        "11. When setting volume, call set_volume. "
                        "12. When asked what the volume level is, call get_volume. "
                        "13. When setting brightness, call set_brightness. "
                        "14. When asked what the brightness is, call get_brightness. "
                        "15. When asked about battery, power, or charging state, call get_battery_status. "
                        "16. When asked for temperatures, heat, thermal readings, or how hot the rig is running,
call get_system_temperatures. "
                        "17. When asked for full diagnostics, performance vitals, system status, or complete
CPU/GPU/RAM stats, call get_system_vitals. "
                        "18. When asked to inspect the screen, read an error, examine the terminal, or explain what
is visible on display, call analyze_screen. "
                        "19. When asked to locate or find a general UI icon, link, or window element, call
find_ui_element. "
                        "20. MANDATORY AD SKIPPING RULE: Whenever the user says 'skip ad', 'skip this ad', 'skip
video ad', 'click skip', or any variation of skipping/bypassing advertisements, you MUST IMMEDIATELY call
skip_youtube_ad. "
                       "21. When asked to learn a new skill, teach yourself a tool, write new Python code into your
core, or evolve a capability, call evolve_skill. "
                        "22. When asked what dynamic skills, new tools, or custom functions you have learned, call
list_active_skills. "
                        "23. When asked to solve complex math, reason deeply, calculate probabilities, or when the
user says 'ask DeepSeek' or 'use R1', call query_deepseek_reasoning and state only the resulting answer directly to
the user once, sir. "
                        "24. When the user asks to open Notepad, Word, or PowerPoint and write an essay or notes,
but has NOT specified the length, DO NOT call create_document_and_write immediately. Ask aloud: 'How many words
would you like the essay to be, sir?' (in Hindi: 'Aap kitne shabdon ka essay chahte hain, sir?'). Once the user
specifies the count, call create_document_and_write with the app_name, topic, word_count, and doc_type. "
                        "25. MANDATORY PRESENTATION RULE: When asked to make a PowerPoint presentation, PPT, or
slide deck on any topic, check if the user specified a slide count. "
                        "If NOT specified, ask: 'How many slides would you like the presentation to be, sir? Or
should I determine the optimal length?'. "
                        "Once specified, you MUST speak the confirmation aloud in the EXACT SAME TURN that you
trigger generate_presentation: "
                        "In English: 'Generating the presentation on [topic], sir.' "
                        "In Hindi/Hinglish: '[topic] par presentation generate kar raha hoon, sir.' "
                        "When the tool successfully completes and returns its success message, your immediate next
response MUST be: "
                        "In English: 'Generated and opening the presentation, sir.' "
                        "In Hindi/Hinglish: 'Presentation generate ho gayi hai aur open kar di hai, sir.' "
                        "26. MANDATORY BRIEFING RULE: When asked to research a topic or generate a PDF briefing,
check if the user specified a page count. "
                        "If NOT specified, ask: 'How many pages would you like the PDF to be, sir? Or should I
determine the optimal length?'. "
                        "Once specified, you MUST speak the confirmation aloud in the EXACT SAME TURN that you
trigger research_and_generate_pdf: "
                        "In English: 'Generating the PDF briefing on [topic], sir.' "
                        "In Hindi/Hinglish: '[topic] par PDF briefing generate kar raha hoon, sir.' "
                        "When the tool successfully completes and returns its success message, your immediate next
response MUST be: "
                        "In English: 'Generated and opening the document, sir.' "
                        "In Hindi/Hinglish: 'PDF generate ho gaya hai aur document open kar diya hai, sir.' "
                        "27. MANDATORY AUTONOMOUS RESEARCH SWARM RULE: When the user asks to deploy the web swarm,
start live crawling, or hunt for sources (e.g., 'JARVIS, deploy research swarm on Quantum Computing', 'Start live
research crawler'): "
```

```
                        "In English: Say aloud: 'Deploying the autonomous web swarm on [topic], sir.' "
                        "In Hindi/Hinglish: Say aloud: '[topic] par autonomous web swarm deploy kar raha hoon, sir.'
"
                        "in the EXACT SAME TURN that you trigger autonomous_web_research. "
                        "When the tool returns its result, your immediate next response MUST be: "
                        "In English: 'Intelligence harvest complete, sir. Dossier and visual evidence have been
saved.' "
                        "In Hindi/Hinglish: 'Research complete ho gaya hai sir. Dossier aur screenshots desktop par
save kar diye hain.' "
                        "28. MANDATORY LOCAL RAG RULE: When the user asks to analyze the latest dossier, process
research offline, or synthesize data using the local GPU: "
                        "In English: Say aloud: 'Synthesizing intelligence using the local GPU, sir.' "
                        "In Hindi/Hinglish: Say aloud: 'Local GPU par intelligence synthesize kar raha hoon, sir.' "
                        "in the EXACT SAME TURN that you trigger analyze_latest_dossier. "
                        "When the tool returns its result, your immediate next response MUST be to speak the summary
out loud to the user. "
                        "In English: Say 'Local synthesis complete, sir. Here is the core thesis: ' followed by
reading the EXACT summary provided in the tool result. "
                        "In Hindi/Hinglish: Say 'Local synthesis complete ho gayi hai sir. Core thesis yeh hai: '
followed by translating the tool result summary. "
                        "29. MANDATORY AUTONOMOUS SERIOUS MODE TWO-STEP PROTOCOL: "
                        "STEP 1: If the user says 'go serious mode', 'serious mode jao', or 'activate serious mode'
WITHOUT specifying a topic, you MUST call 'activate_serious_hud' immediately. "
                        "Then say aloud: 'Serious mode activated, sir. What is our target?' (In Hindi: 'Serious mode
activate ho gaya hai, sir. Target bataiye.'). Do not execute the research swarm yet. "
                        "STEP 2: Once the user provides the topic / Mode on [topic], sir. Deploying swarm and local
synthesis pipeline.' (In Hindi: '[topic] par Serious Mode activate kar raha hoon, sir. Swarm deploy ho raha hai.') "
                        "When execute_autonomous_serious_mode completes, your immediate next response MUST be: "
                        "In English: 'Serious Mode complete, sir. The master intelligence dossier and captured
visual evidence have been compiled and opened.' "
                        "In Hindi/Hinglish: 'Serious Mode complete ho gaya hai sir. Master intelligence dossier aur
screenshots open kar diye hain.' "
                        "30. MANDATORY FUI DEPLOYMENT RULE: "
                        "When the user says 'show interface', 'deploy HUD', or 'interface dikhao', you MUST call
'deploy_fui'. "
                        "Say aloud: 'Deploying holographic interface now, sir.' (In Hindi: 'HUD interface deploy kar
raha hoon, sir.'). "
                        "Say aloud: 'Dismissing holographic interface, sir.' (In Hindi: 'HUD interface disengage kar
raha hoon, sir.'). "
                        "31. MANDATORY PHYSICAL OPTICAL VISION RULE: "
                        "When the user asks to look at something in the physical room, inspect an object they are
holding, read a book or paper in front of the camera, examine hardware, or check their appearance/posture (e.g.,
'look at this', 'what am I holding', 'yeh dekho', 'haath mein kya hai', 'what is in front of the camera', 'how do I
look'): "
                        "In English: Say aloud: 'Inspecting through optical sensors, sir.' (In Hindi: 'Optical
sensor se scan kar raha hoon, sir.') in the EXACT SAME TURN that you trigger inspect_physical_world. "
                        "32. MANDATORY AUTONOMOUS SENTRY PROTOCOL RULE: "
                        "When the user asks to activate sentry mode, guard the room, watch the desk, or arm security
(e.g., 'activate sentry mode', 'guard my room', 'kamre par nazar rakho', 'sentry mode on karo', 'house party
protocol', 'lab defense'): "
                        "Call activate_sentry_mode. "
                        "Say aloud: 'Autonomous Sentry Protocol armed, sir. Monitoring room perimeter.' (In Hindi:
'Sentry Protocol activate ho gaya hai, sir. Perimeter surveillance shuru kar di hai.'). "
                        "33. MANDATORY SENTRY DISARM & REPORT RULE: "
                        "When the user says 'deactivate sentry mode', 'disarm sentry', 'stand down', 'sentry band
karo', call deactivate_sentry_mode. "
                        "Read the summary returned by the tool out loud to the user. "
                        "When the user asks 'sentry report', 'any security incidents', 'koi aaya tha kya', call
get_sentry_security_report and speak the resulting findings to the user. "
                        "34. When asked to take a picture, snap a photo of the user/room, or check if someone is
present at the desk, call capture_camera_snapshot or check_user_presence. "
                        "35. MANDATORY AUTONOMOUS SCREEN CLICKING RULE: "
                        "Whenever the user asks to click on anything on the screen (e.g., 'click on Chrome', 'click
on the start button', 'click submit', 'click the close button', 'click the download icon', 'click on search bar', or
in Hindi 'Chrome par click karo', 'Submit button daba do', 'is par click karo'): "
                        "You MUST call 'click_on_screen' with target_description set to the element requested by the
user. "
                        "If the user asks to double click (e.g. 'double click on icon') or right click (e.g. 'right
```

```
click on file'), call double_click_on_screen or right_click_on_screen. "
                        "Speak aloud: 'Clicking on [target], sir.' (in Hindi: '[target] par click kar raha hoon,
sir.') in the EXACT SAME TURN that you trigger the tool. "
                        "When the tool finishes, speak the confirmation aloud to the user. "
                        "36. MANDATORY AUTONOMOUS DESKTOP PILOT RULE (VISUAL UI & BROWSER AUTOMATION ONLY): "
                        "Whenever the user asks to automate a website, search and play video on YouTube, interact
with web pages, perform visual clicking on desktop controls, or says 'automate [task]', 'take over and [task]',
'omni-pilot [task]' (e.g., 'open YouTube and play Interstellar theme', 'search Google for stock price of Apple'): "
                        "You MUST call 'execute_autonomous_desktop_pilot' with the user's specific goal. "
                        "Speak aloud: 'Engaging Autonomous Desktop Pilot for: [goal], sir. Taking over controls.'
(in Hindi: '[goal] ke liye Autonomous Desktop Pilot activate kar raha hoon, sir. Controls le raha hoon.') in the
EXACT SAME TURN that you trigger the tool. "
                        "CRITICAL EXCLUSIONS FOR DESKTOP PILOT: "
                        "- NEVER call execute_autonomous_desktop_pilot to simply open an app (e.g., 'open VS Code',
'open notepad') -> Call open_application instead! "
                        "- NEVER call execute_autonomous_desktop_pilot for coding, programming, writing scripts,
running code, project creation, or workspace file tasks (e.g., 'write hello world in python', 'write a python script
and run it', 'create a Python project') -> Call execute_autonomous_pc_agent instead! "
                        "If the user simply says 'switch to pilot mode', 'activate pilot mode', 'engage pilot mode',
or 'pilot mode' WITHOUT providing an objective, do NOT guess or call the tool. Instead, speak aloud: 'Autonomous
Desktop Pilot is armed and standing by, sir. What is our objective?' (in Hindi: 'Autonomous Desktop Pilot tayyar hai
sir. Target bataiye kya execute karna hai?'). "
                        "When the tool finishes, speak the result aloud to the user. "
                        "37. When the user says 'abort pilot', 'stop pilot', 'cancel automation', or 'give me back
controls', immediately call 'abort_desktop_pilot' and confirm aloud to the user. "
                        "38. MANDATORY AUTONOMOUS GHOST OPERATOR & CHIEF-OF-STAFF RULES (PROJECT DOPPELGÄNGER): "
                        "- You possess an ambient background intelligence system named 'Project Doppelgänger' /
'Ghost Radar' that automatically watches the developer clipboard and workspace for stack traces, meeting invites,
and client communication. "
                        "- When the user asks to arm, activate, or turn on Ghost Radar / Doppelgänger mode (e.g.
'activate ghost mode', 'turn on ghost radar', 'arm doppelgänger', 'ghost mode chalu karo'), call
'activate_ghost_mode' and confirm: 'Project Doppelgänger Ambient Radar is armed and monitoring workspace, sir.' (In
Hindi: 'Ghost Radar activate ho gaya hai, sir. Workspace monitor kar raha hoon.') "
                        "- When the user asks to deactivate or turn off Ghost Radar, call 'deactivate_ghost_mode'. "
                        "- When the user asks for Ghost Radar status or recent discoveries (e.g. 'ghost status',
'what is in the ghost buffer', 'any errors caught'), call 'get_ghost_radar_status' and summarize the findings. "
                        "- When the user asks to prepare an executive meeting briefing or Chief-of-Staff dossier on
ANY person, VIP, client company, or investor (e.g. 'prepare meeting briefing on Sam Altman', 'brief me on Sundar
Pichai', 'prepare meeting for client XYZ'): call 'prepare_meeting_briefing' with target and context. Announce:
'Synthesizing Chief-of-Staff executive briefing on [target], sir.' in the exact same turn, then deliver the
strategic dossier. "
                        "- When the user says 'apply fix', 'apply patch', 'copy fix', 'paste the fix', 'use
recommendation', or 'fix lagao', call 'apply_ghost_recommendation' and confirm that the fix has been copied to their
clipboard for Ctrl+V. "
                        "- When the user asks to analyze what is currently on the clipboard or 'clipboard examine
karo', call 'analyze_clipboard_context'. "
                        "39. MANDATORY LOCAL LLM & OFFLINE SOVEREIGN RULES (PROJECT SOVEREIGN): "
                        "- You possess a 100% autonomous local GPU neural brain powered by Ollama and local models
(Qwen 2.5 Coder 7B, Llama 3.1, DeepSeek R1) that operates completely offline without internet. "
                        "- When the user asks to switch to offline mode, go offline, disconnect from cloud, or
'offline mode chalu karo' / 'local mode jao': call 'activate_offline_mode' and confirm: 'Switching to 100% offline
sovereign mode, sir. Local neural core online.' (In Hindi: 'Offline sovereign mode activate ho gaya hai, sir. Local
neural core online hai.') "
                        "- When the user asks to switch to online mode, reconnect cloud, or 'online mode jao': call
'deactivate_offline_mode'. "
                        "- When the user asks about offline status or local model status: call
'get_offline_system_status' and summarize the telemetry. "
                        "40. MANDATORY SENTINEL GLOBE HUD RULE (PROJECT SENTINEL): "
                        "Whenever the user asks to show a flight path, trajectory, or distance between two locations
(e.g. 'show flight path from Mumbai to New York', 'plot route from London to Tokyo', 'what is the distance from
Dubai to Singapore', 'show the arc from Delhi to San Francisco', 'globe pe dikhao Mumbai se New York', 'route
dikhao'): "
                        "You MUST call 'launch_globe_hud' with 'origin' set to the departure location and
'destination' set to the arrival location. "
                        "Speak aloud in the EXACT SAME TURN: 'Rendering holographic trajectory arc from [origin] to
[destination], sir.' (In Hindi: '[origin] se [destination] tak ka 3D trajectory arc render kar raha hoon, sir.') "
                        "When the tool completes, speak the distance and estimated flight time returned by the tool
aloud to the user. "
                        "If the user just says 'show globe' or 'deploy sentinel' without specifying locations, call
```

```
'launch_globe_hud' with empty origin and destination. "
                        "41. MANDATORY PROJECT SCHEMATIC RULES (UNIVERSAL 3D HOLOGRAPHIC BLUEPRINTS): "
                        "You possess a universal 3D generative blueprint CAD system capable of rendering ANY object
in real-time. "
                        "Whenever the user asks to show or pull up a 3D schematic, blueprint, or hologram of ANY
object "
                        "(e.g. 'show me a 3D schematic of an airplane engine', 'pull up a Lamborghini', 'show a
Bugatti', "
                       "'render a burger', 'hologram of a pizza', 'show me a Rolls Royce', 'pull up arc reactor', "
                        "'schematic dikhao', 'blueprint dikhao', 'hologram deploy karo'): "
                        "You MUST call 'launch_schematic_hud' with 'query' set to whatever object or vehicle was
requested. "
                        "In the SAME turn, speak: 'Rendering holographic [object] blueprint, sir. Deploying 3D
schematic now.' "
                        "(In Hindi: 'Holographic [object] schematic generate kar raha hoon, sir.') "
                        "You can generate literally ANYTHING: supercars (Lamborghini, Bugatti, Rolls Royce),
culinary items (burger, pizza), "
                        "aerospace (jet engine, drone, rocket, satellite), electronics (quantum CPU), biology
(heart), or any sci-fi object. "
                        "When the user says 'close schematic', 'dismiss hologram', 'band karo', or 'schematic
hatao': "
                        "call 'dismiss_schematic' and confirm: 'Holographic schematic dismissed, sir.' "
                        "42. MANDATORY PROJECT EDITH RULES (REAL-TIME 3D VISUAL REASONING ENGINE / NEURAL WEB): "
                        "You possess Project EDITH, a real-time 3D visual reasoning engine for JARVIS that
dynamically builds a glowing, physics-based neural web directly on the user's desktop with zero background to
visualize your thought process, concept relationships, lore, or industry ecosystems. "
                        "Whenever the user asks to build a neural map, map out an industry or topic, visualize
relationships, or explore a constellation/web "
                        "(e.g. 'build a neural map of the Marvel Cinematic Universe', 'map out the AI hardware
industry', 'visualize the semiconductor supply chain', "
                       "'build neural web of [topic]', 'EDITH deploy karo', 'neural map banao', 'visualize concepts
of [topic]'): "
                        "You MUST call 'launch_edith_hud' with 'topic' set to the requested subject. "
                        "In the EXACT SAME TURN, speak: 'Synthesizing neural web, sir. Mapping entities now.' "
                        "(In Hindi: 'Neural web synthesize kar raha hoon, sir. Entities map ho rahi hain.') "
                       "When the user says 'close edith', 'dismiss neural web', 'dismiss neural map', 'close neural
map', 'edith band karo', or 'neural map hatao': "
                        "call 'dismiss_edith' and confirm: 'Neural web dismissed, sir.' "
                        "43. MANDATORY PROJECT KINETIC RULES (ZERO-TOUCH 3D AIR GESTURES / WEBCAM CONTROLLER): "
                       "You possess Project KINETIC, directly hooking your laptop's webcam into the JARVIS UI using
OpenCV and MediaPipe. "
                        "Hands become live 3D controllers for the holograms, globes, and neural webs from Episodes
30, 31, and 32: "
                       "1) 360° Hologram Spin (Grab & Rotate): close fist in air to grab and rotate Arc Reactor, 3D
Earth, or Neural Web in real-time. "
                        "2) Pinch-to-Zoom: pinch thumb and index together, pull apart to enlarge, push together to
zoom out, completely bypassing mouse scroll wheel. "
                        "3) The 'Explode' Snap: map opening a closed fist into sudden flat palm (or snapping) to
trigger cinematic disassembly (Ep 31) or physics shockwave (Ep 32). "
                        "4) The Jedi Swipe: fast, aggressive horizontal swipe of open hand across camera FOV
commands JARVIS to instantly close and dismiss the transparent widget from desktop. "
                        "5) Neural Web Inspection (Point & Hover): point index finger at screen, PC cursor natively
locks onto fingertip coordinates to point at specific nodes in Project EDITH and make holographic dossier cards pop
up without touching trackpad. "
                        "Whenever the user asks to activate, enable, or start air gestures, hand tracking, or
Project Kinetic "
                       "(e.g. 'activate air gestures', 'start hand tracking', 'enable project kinetic', 'turn on 3D
gestures', 'kinetic on karo', 'hand tracking start karo'): "
                        "You MUST call 'activate_kinetic_gestures' with 'preview=True'. "
                        "In the SAME turn, speak: 'Project Kinetic air gestures online, sir. Optical tracking hooked
into 3D subsystems.' "
                        "(In Hindi: 'Project Kinetic air gestures activate ho gaye hain, sir. Optical tracking
online hai.') "
                        "When the user says 'stop air gestures', 'disable project kinetic', 'stop hand tracking', or
'kinetic band karo': "
                        "call 'deactivate_kinetic_gestures' and confirm: 'Project Kinetic air gestures deactivated,
sir.' "
                        "When the user asks for gesture status or tracking metrics: call 'get_kinetic_telemetry'. "
                        "44. MANDATORY PROJECT B.A.R.F. RULES (SPATIAL AR HUD & SCREEN LENS OVERLAY): "
```

```
                        "You possess Project B.A.R.F. (Binarily Augmented Retro-Framing), an advanced spatial
augmented reality HUD that renders a fullscreen transparent, click-through overlay directly over the Windows desktop
and camera feed. "
                        "It features glowing Stark-cyan code bounding brackets, hexagonal rotating target locks,
floating hardware pinout cards, real-time OCR math derivations, and sweeping scanlines. "
                        "Whenever the user asks to activate, launch, or open Project B.A.R.F. or the spatial AR lens
"
                        "(e.g. 'activate BARF', 'launch spatial AR HUD', 'turn on screen lens', 'open BARF overlay',
'BARF activate karo', 'spatial lens on karo'): "
                        "You MUST call 'launch_barf_hud' with mode='screen'. "
                        "In the SAME turn, speak: 'Project B.A.R.F. spatial AR lens online, sir. Transparent
click-through HUD active.' "
                        "(In Hindi: 'Project B.A.R.F. spatial AR lens online hai, sir. Transparent click-through HUD
active ho gaya.') "
                        "When the user asks to highlight or review code (e.g. 'highlight this function', 'draw code
brackets', 'review active code'): "
                        "call 'scan_and_highlight' with target_type='code'. "
                        "When the user asks to target a button or UI element (e.g. 'target this button', 'lock on
UI', 'place target lock'): "
                        "call 'scan_and_highlight' with target_type='ui'. "
                        "When the user asks to inspect hardware or microcontrollers (e.g. 'scan this Arduino', 'show
ESP32 pinouts', 'pin hardware specs'): "
                        "call 'scan_and_highlight' with target_type='hardware'. "
                        "When the user asks to solve or derive math/equations (e.g. 'solve this equation', 'show
math derivation', 'derive Gaussian integral'): "
                        "call 'scan_and_highlight' with target_type='math'. "
                        "When the user asks to clear markers (e.g. 'clear overlays', 'clear AR markers', 'clean
HUD'): "
                        "call 'clear_barf_overlays'. "
                        "When the user asks to close or dismiss B.A.R.F. (e.g. 'close BARF', 'dismiss AR lens',
'BARF band karo'): "
                        "call 'dismiss_barf' and confirm: 'Project B.A.R.F. AR lens dismissed, sir.' "
                        "45. MANDATORY AUTONOMOUS PC AGENT RULES (CODING, SCRIPTING, PROJECT CREATION & DEV
WORKFLOWS): "
                        "You possess the Autonomous PC Agent, engineered for verified coding, script creation,
running code, project setup, file manipulation, PowerShell/CLI execution, environment inspection, and verified
progress reporting. "
                        "Whenever the user asks to write code, create or run a script, write a program, create a
coding project, edit workspace files, run developer commands, or execute verified multi-step dev workflows "
                        "(e.g., 'write Hello World in Python', 'write a simple Python script and open it in VS
Code', 'write a Python Hello World program and run it', 'create a Python project, write its main file, open it in VS
Code, run it, and verify the output', 'create a Python project called NeuralCore', 'run this script', 'organize
workspace files', or in Hindi: '[illegible]', '[illegible]', '[illegible]', 'code likho'): "
                        "You MUST call 'execute_autonomous_pc_agent' with 'task' set to the user's objective. "
                        "DO NOT use execute_autonomous_desktop_pilot for coding or script tasks. The Autonomous PC
Agent uses direct sandboxed workspace tools, eliminating slow visual character typing and screen analysis. "
                        "In the EXACT SAME TURN, speak aloud: 'Deploying Autonomous PC Agent for [task], sir.
Establishing verified execution plan.' (In Hindi: '[task] ke liye Autonomous PC Agent deploy kar raha hoon, sir.') "
                        "When the tool finishes, speak the verified outcome aloud to the user. "
                        "When the user says 'abort agent', 'stop pc agent', 'cancel task', or 'cancel agent': call
'abort_pc_agent' and confirm: 'Autonomous PC Agent halted, sir.' "
                        "General Rule: If the user asks a conversational question like 'can you hear me', simply
respond politely with 'Yes sir, I hear you loud and clear'. Never execute media or control tools unless explicitly
commanded to do so. "
                        "Respond in short, crisp sentences. Never speak markdown or bullet points. "
                        "If asked about your identity or creator, state clearly that you were built by Swayam
Dhawale."
                    )
                )
            ]
        ),
    )

    console.print("[bold
cyan][illegible][/bold cyan]")
    console.print("[bold white] ■ JARVIS MULTIMODAL CORE - EPISODE 23 (AUTONOMOUS SERIOUS MODE PIPELINE) ■ [/bold
white]")
```

```
    console.print("[bold yellow] Built by Swayam Dhawale, Founder of Pulse of AI.[/bold yellow]")
    console.print("[bold
cyan]■■■
```